TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH

TRUNG TÂM LOGISTICS

ĐỀ TÀI

XÂY DỰNG MÔ HÌNH TỐI ƯU HÓA TUYẾN ĐƯỜNG VÀ GIẢI QUYẾT XUNG ĐỘT GIỮA CÁC ROBOT

MÔN HỌC: SMART LOGISTICS

Giảng viên hướng dẫn: TS. Trịnh Tuấn Hùng

Sinh viên thực hiện:

1. Trần Chí Tường – 089207015516
2. Đoàn Phạm Lan Trinh – 049305008786
3. Nguyễn Yến Nhi – 049305004288
4. Phạm Nguyễn Nghi Linh – 077304000570
5. Ngô Trung Hiếu – 045205004181

MỤC LỤC

MỤC LỤC **2**

DANH MỤC CÁC BẢNG **6**

DANH MỤC CÁC HÌNH, SƠ ĐỒ **7**

LỜI MỞ ĐẦU **8**

Tính cấp thiết của đề tài 8

Tổng quan về tình hình nghiên cứu 8

Mục tiêu nghiên cứu 8

Đối tượng và phạm vi nghiên cứu 8

Phương pháp nghiên cứu 8

Ý nghĩa khoa học và thực tiễn 9

Kết cấu tiểu luận 9

CHƯƠNG 1: CƠ SỞ LÝ LUẬN **10**

1.1. Tổng quan về nhà kho thông minh 10

1.1.1. Khái niệm và các mô hình nhà kho ***10***

1.1.2. Số hóa, tự động hóa và thiết bị thông minh trong kho ***10***

1.2. Robot tự hành AGV trong nhà kho 10

1.2.1. Khái niệm và nguyên lý hoạt động của AGV ***10***

1.2.2. Vai trò và những vấn đề khi nhiều AGV cùng hoạt động ***10***

1.3. Cơ sở lý thuyết về khoảng cách và tìm đường 10

1.3.1. Biểu diễn không gian kho và tính khoảng cách ***10***

1.3.2. Thuật toán tìm đường Dijkstra và A* ***11***

1.4. Xung đột và phương pháp tránh va chạm giữa các robot 11

1.4.1. Các tình huống xung đột trong quá trình vận hành ***11***

1.4.2. Dự đoán va chạm, điều chỉnh vận tốc và tìm đường lại ***11***

CHƯƠNG 2: MÔ HÌNH VÀ KIẾN TRÚC HỆ THỐNG **12**

2.1. Mô hình mục tiêu 12

2.2. Cấu trúc logic của mô hình 12

2.2.1. Lớp dữ liệu *12*
2.2.2. Lớp xử lý nghiệp vụ *12*
2.2.3. Lớp tối ưu hóa tuyến đường *13*
2.2.4. Lớp phòng tránh va chạm *13*
2.2.5. Lớp hiển thị và giám sát *13*
CHƯƠNG 3: XÂY DỰNG MÔ HÌNH TỐI ƯU TUYẾN ĐƯỜNG VÀ GIẢI QUYẾT CÁC XUNG ĐỘT **14**
Mục tiêu xây dựng mô hình 14
Kiến trúc tổng thể của mô hình 15
3.3. Cài đặt thư viện và công cụ 17
3.4. Thiết kế và tiền xử lý dữ liệu 17
3.4.1. Nguồn dữ liệu Grocery Inventory *17*
3.4.2. Tiền xử lý dữ liệu và phân loại FEFO *17*
3.5. Xây dựng mô hình tối ưu tuyến đường 17
3.5.1. Mục tiêu và tổ chức vận hành *17*
3.5.2. Không gian kho và mô hình lưới *18*
3.5.3. Dijkstra, A và cơ chế tránh robot* *19*
3.5.4. Chu trình PICK DROP RETURN và quy tắc điều hướng *20*
3.6. Kiểm soát xung đột và va chạm 21
3.6.1. Kiến trúc nhiều lớp và dự đoán CPA *21*
3.6.2. Điều tốc theo phanh và xác nhận bước di chuyển *22*
3.6.3. Phục hồi, quét 360° và thứ tự cập nhật *24*
3.6.4. Quy tắc cảm biến và điều tốc *24*
3.6.5. Quy tắc phòng tránh và phục hồi *26*
3.6.6. Ưu điểm và giới hạn kiểm soát an toàn *26*
3.7. Giao diện mô phỏng và giám sát 27
3.7.1. Công nghệ, không gian 3D và camera *27*
3.7.2. Tương tác FPS và Mini-map *28*
3.7.3. Tablet WMS, hàng đợi và ô sạc *29*

3.7.4. Vòng lặp cập nhật, đo thời gian và đánh giá FPS *31*

CHƯƠNG 4: QUY TRÌNH VẬN HÀNH **32**

4.1. Thiết lập môi trường vận hành 32

4.2. Chuẩn hóa và xử lý dữ liệu 32

4.3. Thứ tự thực thi chống va chạm 33

CHƯƠNG 5: THÍ NGHIỆM VÀ ĐÁNH GIÁ **34**

5.1. Thiết lập thí nghiệm 34

5.1.1. Cấu hình và phạm vi thử nghiệm ***34***

5.1.2. Thiết kế bảy kịch bản ***34***

5.1.3. Phương pháp đo và chỉ số ***34***

5.1.4. Quy ước ghi nhận ***35***

5.2. Đánh giá kết quả thí nghiệm 35

5.2.1. Tổng hợp kết quả TN01–TN07 ..... ***35***

5.2.2. Đánh giá thời gian và năng suất ***35***

5.2.2.1. Phân tích sai lệch và thông lượng ***36***

5.2.3. Đánh giá nhường đường, an toàn và phục hồi ***37***

5.2.4. Phân tích các kịch bản đặc trưng ***37***

5.2.5. Phân tích khoảng cách an toàn ***38***

5.2.6. Hình ảnh kết quả thử nghiệm ***38***

5.3. Đánh giá tổng quan mô hình 39

5.3.1. Vấn đề giải quyết được ***39***

5.3.2. Cách giải quyết vấn đề ***39***

5.3.3. Vấn đề chưa giải quyết được ***40***

5.3.4. Phạm vi áp dụng kết luận ***40***

5.4. Định hướng phát triển 40

5.4.1. Giai đoạn ngắn hạn – hoàn thiện kiểm chứng ***40***

5.4.2. Giai đoạn trung hạn – tối ưu điều phối ***40***

5.4.3. Giai đoạn dài hạn – tiếp cận thực tế ***41***

5.4.4. Ma trận ưu tiên thực hiện ***41***

5.4.5. Đề xuất thí nghiệm tiếp theo 41
5.5. Hồ sơ kiểm chứng 41
5.5.1. /exam và quy trình kiểm thử 41
5.5.2. /logs và schema sự kiện 41
5.5.3. /evals và tiêu chí đánh giá 41
CHƯƠNG 6: KẾT LUẬN VÀ KIẾN NGHỊ 43
6.1. Kết luận 43
6.2. Định hướng phát triển 43
TÀI LIỆU THAM KHẢO 43

DANH MỤC CÁC BẢNG

STT	Tên Bảng	Trang
1	Bảng 1.1: Một số cột mốc chính trong quá trình hình thành và phát triển của Công ty Cổ phần Logistics U&I.	

DANH MỤC CÁC HÌNH, SƠ ĐỒ

STT	Tên hình	Trang
1	Hình 3.1. Sơ đồ kiến trúc tổng thể mô hình kho FEFO 3D với sáu robot AGV.	18

LỜI MỞ ĐẦU

Tính cấp thiết của đề tài

Trong kho hàng, việc sử dụng nhiều robot tự hành AGV (Automated Guided Vehicle) giúp giảm công việc vận chuyển và lấy hàng thủ công. Tuy nhiên, khi các robot cùng di chuyển trong không gian có nhiều kệ hàng và lối đi hẹp, chúng có thể gặp nhau tại các giao lộ, cản đường hay phải đứng chờ, làm tăng thời gian thực hiện đơn hàng. Vì vậy, nhóm đã chọn đề tài "Tối ưu hoá tuyến đường và giải quyết xung đột trong vận hành Robot tự hành (AGV) trong nhà kho thông minh" để nghiên cứu cách tìm đường và xử lý xung đột giữa các robot.

Tổng quan về tình hình nghiên cứu

Dijkstra và A* là những thuật toán được sử dụng để tìm đường đi ngắn nhất trên mạng lưới có chướng ngại vật. Tuy nhiên, khi nhiều robot cùng hoạt động, tìm được đường đi ngắn chưa đủ để tránh xung đột. Vì vậy, nhóm đã sử dụng thuật toán A* kết hợp với cơ chế dự đoán nguy cơ va chạm CPA (Closest Point of Approach), điều chỉnh vận tốc và tìm đường lại để xử lý các tình huống phát sinh.

Mục tiêu nghiên cứu

Xây dựng mô hình kho thông minh 3D với 5 robot AGV có khả năng tìm đường kết hợp cơ chế tránh xung đột và xử lý nhiều đơn hàng liên tiếp. Đồng thời, nhóm áp dụng nguyên tắc FEFO (First Expired, First Out) để phân loại hàng hoá theo thời hạn sử dụng. Qua mô hình, nhóm hướng đến việc đánh giá thời gian di chuyển, quãng đường, tình trạng xung đột và khả năng thực hiện nhiệm vụ của các robot.

Đối tượng và phạm vi nghiên cứu

Đề tài nghiên cứu việc tìm đường và xử lý xung đột giữa các robot AGV trong kho. Mô hình được xây dựng trên không gian kho giả lập 28 x 28m với 5 robot AGV, tập trung vào quá trình lấy hàng, giao hàng và quay về trạm sạc. Nghiên cứu được thực hiện trên phần mềm mô phỏng, chưa được áp dụng trên robot thực tế.

> Phạm vi cấu hình: notebook hợp nhất hiện dùng 5 AGV. Các số liệu TN01–TN07 trong Chương 5 được giữ nguyên theo baseline V21 6 AGV để đối chiếu; đó không phải là số đo mới của cấu hình 5 AGV.

Phương pháp nghiên cứu

Nhóm kết hợp phương pháp nghiên cứu tài liệu, mô hình hoá, lập trình mô phỏng và phân tích kết quả. Dữ liệu Grocery Inventory từ Kaggle được xử lý bằng Python; môi trường kho 3D và hoạt động robot được xây dựng bằng JavaScript và Three.js. Thuật toán A* được sử dụng để tìm đường, kết hợp CPA và cơ chế điều chỉnh tốc độ để xử lý nguy cơ xung đột. Kết quả được xem xét thông qua quãng đường, số lần xung đột và chênh lệch giữa thời gian dự tính theo công thức ($t=s/v$; $v = 2{,}3$ m/s) và thời gian thực hiện ghi nhận trong mô phỏng.

Ý nghĩa khoa học và thực tiễn

Về khoa học: Đề tài giúp tìm hiểu cách kết hợp thuật toán A*, cơ chế dự đoán va chạm và điều chỉnh tốc độ trong mô hình nhiều robot AGV hoạt động đồng thời.

Về thực tiễn: Mô hình 3D cho phép quan sát quá trình robot di chuyển, lấy hàng và xử lý xung đột, từ đó nhận biết những hạn chế và đề xuất giải pháp cải tiến hệ thống trong kho.

Kết cấu tiểu luận

Ngoài phần lời mở đầu và tài liệu tham khảo, tiểu luận gồm 6 chương:

Chương 1: Cơ sở lý luận.

Chương 2: Giới thiệu mô hình tối ưu hóa tuyến đường và xử lý xung đột giữa các robot.

Chương 3: Xây dựng mô hình tối ưu tuyến đường và giải quyết các xung đột.

Chương 4: Quy trình vận hành mô hình.

Chương 5: Thí nghiệm và đánh giá mô hình.

Chương 6: Kết luận và kiến nghị.

CHƯƠNG 1: CƠ SỞ LÝ LUẬN

1.1. Tổng quan về nhà kho thông minh

1.1.1. Khái niệm và các mô hình nhà kho

Kho hàng là nơi tiếp nhận, lưu trữ, bảo quản và chuẩn bị hàng hóa trước khi đưa đến các khâu tiếp theo trong chuỗi cung ứng. Dựa trên mức độ ứng dụng công nghệ, có thể phân biệt ba mô hình: kho truyền thống, kho tự động hóa và kho thông minh.

Kho truyền thống chủ yếu dựa vào nhân công, kho tự động hóa sử dụng thêm máy móc, băng tải hoặc robot để hỗ trợ công việc, còn kho thông minh (Smart Warehouse) kết hợp thiết bị tự động với hệ thống quản lý và dữ liệu để theo dõi, điều phối hoạt động kho [1].

1.1.2. Số hóa, tự động hóa và thiết bị thông minh trong kho

Số hóa giúp lưu trữ và xử lý thông tin hàng hóa bằng phần mềm, còn tự động hóa giúp giảm thao tác thủ công. Hệ thống quản lý kho WMS (Warehouse Management System) hỗ trợ theo dõi tồn kho, vị trí lưu trữ và hoạt động nhập, xuất hàng. Các công nghệ như mã vạch, RFID, cảm biến và robot hỗ trợ nhận diện, theo dõi và vận chuyển hàng hoá [2].

Đối với hàng hóa có hạn sử dụng, nguyên tắc FEFO (First Expired, First Out) được sử dụng để ưu tiên xuất sản phẩm có hạn sử dụng gần nhất, nhằm hạn chế hàng hết hạn trong kho.

1.2. Robot tự hành AGV trong nhà kho

1.2.1. Khái niệm và nguyên lý hoạt động của AGV

AGV (Automated Guided Vehicle) là phương tiện vận chuyển tự động, di chuyển theo hệ thống dẫn đường hoặc lộ trình điều khiển. Trong kho, robot có thể nhận nhiệm vụ, di chuyển đến vị trí lấy hàng, đưa hàng đến điểm giao và quay về trạm sạc. Để thực hiện các nhiệm vụ này, robot cần xác định đường đi và tránh các khu vực có chướng ngại vật.

1.2.2. Vai trò và những vấn đề khi nhiều AGV cùng hoạt động

AGV giúp giảm công việc vận chuyển lặp lại và hỗ trợ hoạt động lấy, giao hàng. Nhưng khi nhiều robot cùng hoạt động đồng thời, chúng có thể bị xung đột, làm tăng thời gian chờ. Vì vậy, hệ thống cần kết hợp thuật toán tìm đường và cơ chế xử lý xung đột.

1.3. Cơ sở lý thuyết về khoảng cách và tìm đường

1.3.1. Biểu diễn không gian kho và tính khoảng cách

Để xác định đường đi, không gian kho có thể được mô hình hóa thành bản đồ lưới, gồm các ô có thể di chuyển và các ô bị chặn bởi kệ hàng hoặc vật cản [3].

Hai phương pháp tính khoảng cách thường được sử dụng là:

Khoảng cách Euclidean: Khoảng cách đường thẳng giữa hai điểm.

$d_E = \sqrt{(x_2 - x_1)^2 + (z_2 - z_1)^2}$

Khoảng cách Manhattan: Tổng khoảng cách theo hai trục tọa độ, phù hợp với bản đồ lưới di chuyển theo bốn hướng.

$d_M = |x_2 - x_1| + |z_2 - z_1|$

Trong đó, (x_1, z_1) và (x_2, z_2) là tọa độ của hai vị trí. Khi có chướng ngại vật, quãng đường robot thực hiện có thể dài hơn khoảng cách hình học nên cần thuật toán để xác định tuyến đi phù hợp.

*1.3.2. Thuật toán tìm đường Dijkstra và A**

Dijkstra là thuật toán tìm đường ngắn nhất trên đồ thị có trọng số không âm. Thuật toán lần lượt xem xét những vị trí có chi phí di chuyển thấp nhất để xác định đường đi đến đích [3].

A* là thuật toán tìm đường sử dụng chi phí đã di chuyển kết hợp với chi phí ước lượng đến đích. Hàm đánh giá được biểu diễn như sau [4]: $f(n) = g(n) + h(n)$

Trong đó, g(n) là chi phí từ điểm xuất phát đến vị trí n; h(n) là chi phí ước lượng từ vị trí n đến đích; f(n) là tổng chi phí dự kiến.

Trong kho hàng, A* có thể kết hợp bản đồ chướng ngại vật và chi phí di chuyển để lựa chọn tuyến đường. Khi bổ sung chi phí tránh robot khác, hệ thống có thể ưu tiên các tuyến ít bị cản trở.

1.4. Xung đột và phương pháp tránh va chạm giữa các robot

1.4.1. Các tình huống xung đột trong quá trình vận hành

Xung đột xảy ra khi hai hoặc nhiều robot có nguy cơ sử dụng cùng một vị trí hoặc đoạn đường trong khoảng thời gian gần nhau. Các tình huống thường gặp gồm robot đi ngược chiều trong hành lang hẹp, cùng tiến đến giao lộ hoặc robot tiếp cận quá gần nhau.

Ngoài ra, tình trạng bế tắc (Deadlock) có thể xảy ra khi các robot chờ nhau và không thể tiếp tục di chuyển. Vì vậy, việc tìm được đường cho từng robot cần kèm cơ chế xử lý xung đột.

1.4.2. Dự đoán va chạm, điều chỉnh vận tốc và tìm đường lại

CPA (Closest Point of Approach) là phương pháp dự đoán thời điểm và khoảng cách tiếp cận gần nhất giữa hai robot dựa trên vị trí và vận tốc tương đối, giúp phát hiện sớm nguy cơ va chạm giữa các robot.

Ngoài ra, robot có thể điều chỉnh vận tốc theo khoảng cách còn trống phía trước. Trong điều kiện giảm tốc đều, giới hạn vận tốc theo quãng đường phanh được tính bằng: $v_safe = \sqrt{(2 \times a_brake \times d_free)}$

Trong đó, v_{safe} là vận tốc giới hạn an toàn (m/s); a_{brake} là độ lớn gia tốc phanh (m/s^2); d_{free} là quãng đường còn trống để giảm tốc (m).

Khi phát hiện nguy cơ xung đột, robot có thể giảm tốc, dừng, nhường đường hoặc tìm tuyến thay thế. Việc kết hợp A*, CPA và điều chỉnh vận tốc giúp hệ thống xử lý các tình huống phát sinh. Nhưng các cơ chế này không bảo đảm loại bỏ hoàn toàn xung đột khi nhiều robot hoạt động đồng thời.

CHƯƠNG 2: MÔ HÌNH VÀ KIẾN TRÚC HỆ THỐNG

2.1. Mô hình mục tiêu

Trong bối cảnh chuyển đổi số và phát triển logistics thông minh, các nhà kho hiện đại ngày càng ứng dụng Robot AGV (Automated Guided Vehicle) nhằm tự động hóa hoạt động vận chuyển hàng hóa. AGV là phương tiện vận chuyển tự hành có khả năng di chuyển theo tuyến đã được lập trình hoặc được tính toán động thông qua các thuật toán điều hướng. AGV giúp giảm sự phụ thuộc vào lao động thủ công, nâng cao năng suất làm việc và hạn chế các sai sót trong quá trình vận hành kho.

Mô hình được xây dựng trong đề tài nhằm giải quyết hai bài toán trọng tâm:

Thứ nhất, tối ưu hóa tuyến đường di chuyển của robot từ vị trí xuất phát đến vị trí lấy hàng, sau đó vận chuyển đến điểm giao nhận và quay trở lại trạm sạc.

Thứ hai, giải quyết các tình huống xung đột và nguy cơ va chạm giữa nhiều robot AGV cùng hoạt động trong một không gian kho giới hạn.

Mô hình sử dụng dữ liệu hàng hóa theo nguyên tắc FEFO (First Expired First Out), kết hợp các thuật toán tìm đường và cơ chế tránh va chạm nhằm bảo đảm quá trình vận hành liên tục, an toàn và hiệu quả. FEFO là phương pháp quản lý tồn kho ưu tiên xuất trước các sản phẩm có hạn sử dụng gần nhất nhằm giảm thiểu rủi ro hàng hóa hết hạn.

2.2. Cấu trúc logic của mô hình

Mô hình được xây dựng theo kiến trúc nhiều lớp gồm các thành phần chính:

2.2.1. Lớp dữ liệu

Dữ liệu đầu vào được lấy từ bộ dữ liệu Grocery Inventory trên Kaggle, bao gồm:

- Product ID
- Product Name
- SKU
- Expiration Date
- Stock Quantity

Dữ liệu được xử lý bằng ngôn ngữ Python để loại bỏ dữ liệu lỗi, dữ liệu hết hạn và chuẩn hóa định dạng trước khi đưa vào mô hình.

2.2.2. Lớp xử lý nghiệp vụ

Lớp này thực hiện các chức năng:

- Tính toán ngày còn hạn sử dụng.
- Phân loại hàng hóa theo nguyên tắc FEFO.
- Gán vị trí lưu trữ trong kho.
- Tạo nhiệm vụ cho robot AGV.
- Điều phối và phân bổ công việc.

2.2.3. Lớp tối ưu hóa tuyến đường

Đây là thành phần quan trọng nhất của mô hình.

Thuật toán A* được sử dụng nhằm xác định tuyến đường có chi phí thấp nhất từ vị trí nguồn đến vị trí đích. Thuật toán đánh giá mỗi nút thông qua hàm: f(n) = g(n) + h(n)

Trong đó:

- g(n): chi phí từ điểm xuất phát đến nút hiện tại.
- h(n): chi phí ước lượng từ nút hiện tại tới đích.
- f(n): tổng chi phí cần tối thiểu hóa.

Thuật toán A* hiện là một trong những phương pháp phổ biến trong điều hướng AGV nhờ khả năng tìm đường hiệu quả trong môi trường có vật cản.

2.2.4. Lớp phòng tránh va chạm

Khi nhiều AGV hoạt động đồng thời, nguy cơ xảy ra xung đột tại các giao lộ hoặc hành lang là rất lớn.

Mô hình sử dụng kết hợp:

- Cơ chế dự đoán va chạm CPA (Closest Point of Approach).
- Điều chỉnh vận tốc theo khoảng cách an toàn.
- Tính toán lại lộ trình khi phát hiện vật cản.
- Tăng chi phí đường đi tại các khu vực có robot khác đang hoạt động.

Cách tiếp cận này phù hợp với bài toán Multi-Agent Path Finding (MAPF), trong đó mục tiêu là tối ưu đường đi của nhiều robot đồng thời mà vẫn tránh được xung đột giao thông trong kho.

2.2.5. Lớp hiển thị và giám sát

Hệ thống sử dụng:

- Three.js để mô phỏng kho hàng 3D.
- Dashboard WMS để quản lý nhiệm vụ.
- Mini-map 2D theo dõi vị trí robot.
- Bảng thống kê FEFO.
- Báo cáo lịch sử vận hành.

Thông qua giao diện này người quản lý có thể giám sát toàn bộ hoạt động của kho theo thời gian thực.

CHƯƠNG 3: XÂY DỰNG MÔ HÌNH TỐI ƯU TUYẾN ĐƯỜNG VÀ GIẢI QUYẾT CÁC XUNG ĐỘT

Mục tiêu xây dựng mô hình

Trong bối cảnh hoạt động logistics ngày càng hướng đến tự động hóa và số hóa, việc ứng dụng robot tự hành (AGV – Automated Guided Vehicle) vào quá trình vận chuyển hàng hóa trong kho giúp nâng cao hiệu quả vận hành, giảm sự phụ thuộc vào lao động thủ công và hỗ trợ kiểm soát hoạt động xuất nhập hàng. Tuy nhiên, khi nhiều robot cùng hoạt động trong một không gian kho, các vấn đề như lựa chọn tuyến đường, ùn tắc tại hành lang, xung đột giữa các robot và nguy cơ va chạm có thể ảnh hưởng đến hiệu suất vận hành. Vì vậy, việc xây dựng mô hình mô phỏng kho hàng thông minh với khả năng tối ưu hóa tuyến đường và xử lý xung đột giữa các robot là cần thiết.

Mô hình được xây dựng nhằm mô phỏng hoạt động của một kho hàng thông minh áp dụng nguyên tắc FEFO (First Expired, First Out), kết hợp hệ thống năm robot AGV để thực hiện các nhiệm vụ lấy hàng, vận chuyển đến điểm giao và quay trở về trạm sạc. Trên cơ sở đó, mô hình hướng đến các mục tiêu cụ thể sau:

Thứ nhất, xây dựng hệ thống tìm kiếm và tối ưu hóa tuyến đường di chuyển cho robot AGV. Mô hình sử dụng thuật toán A* trên bản đồ kho dạng lưới để xác định đường đi từ vị trí xuất phát đến vị trí lấy hàng, điểm giao nhận và trạm sạc. Quá trình tìm đường có xét đến các vật cản cố định như hệ thống kệ hàng và các khu vực không được phép di chuyển. Đồng thời, mô hình bổ sung cơ chế tính chi phí đối với những khu vực có robot khác đang hoạt động nhằm ưu tiên các tuyến đường ít có nguy cơ ùn tắc.

Thứ hai, xây dựng cơ chế phát hiện và xử lý xung đột, hạn chế nguy cơ va chạm giữa các robot. Mô hình kết hợp cơ chế dự đoán điểm tiếp cận gần nhất (CPA – Closest Point of Approach), kiểm tra khoảng cách an toàn và kiểm soát tốc độ di chuyển. Khi phát hiện nguy cơ xung đột, robot có thể giảm tốc, tạm dừng hoặc tìm kiếm tuyến đường thay thế. Đối với những trường hợp bị cản trở kéo dài, hệ thống có thể kích hoạt cơ chế quét 360 độ để đánh giá các hướng di chuyển phù hợp.

Thứ ba, mô phỏng hoạt động phối hợp của nhiều robot trong cùng một không gian kho. Mô hình triển khai năm robot AGV có khả năng tiếp nhận và thực hiện các nhiệm vụ riêng thông qua cơ chế hàng đợi. Mỗi robot có thể xử lý nhiều đơn hàng liên tiếp theo thứ tự được phân công, sau khi hoàn thành các nhiệm vụ sẽ tự động quay về trạm sạc. Cơ chế này giúp mô phỏng quá trình vận hành đồng thời của nhiều robot và hỗ trợ quan sát các tình huống phát sinh trong kho.

Thứ tư, xây dựng môi trường mô phỏng trực quan nhằm hỗ trợ theo dõi và đánh giá hoạt động của hệ thống. Thông qua giao diện kho hàng 3D, người dùng có thể quan sát vị trí robot, tuyến đường di chuyển, hệ thống kệ hàng, trạm sạc và trạng thái thực hiện nhiệm vụ. Ngoài ra, hệ thống còn hỗ trợ theo dõi thông tin đơn hàng, tình trạng pin, tốc độ, thời gian thực hiện, quãng đường di chuyển và lịch sử hoạt động của robot.

Nhìn chung, mô hình hướng đến việc kết hợp giữa tối ưu hóa tuyến đường, kiểm soát xung đột và mô phỏng hoạt động của đội robot AGV trong kho hàng thông minh. Qua đó, nhóm có cơ sở quan sát, phân tích và đánh giá khả năng vận hành của hệ thống trong những tình huống khác nhau. Cần lưu ý, sản phẩm hiện tại là mô hình mô phỏng bằng phần mềm, chưa phải hệ thống điều khiển robot vật lý trong kho thực tế.

Kiến trúc tổng thể của mô hình

Kiến trúc tổng thể của mô hình được thiết kế theo hướng kết hợp giữa xử lý dữ liệu bằng Python và mô phỏng, điều phối robot bằng JavaScript. Hệ thống sử dụng dữ liệu hàng hóa từ bộ dữ liệu Grocery Inventory trên Kaggle làm đầu vào, sau đó tiến hành xử lý, chuẩn hóa và chuyển đổi dữ liệu để phục vụ hoạt động mô phỏng kho hàng 3D. Các thuật toán tìm đường, điều phối và xử lý xung đột được triển khai trong môi trường JavaScript, kết hợp với thư viện Three.js để trực quan hóa hoạt động của năm robot AGV.

Về cơ bản, kiến trúc mô hình gồm năm thành phần chính:

(1) Thành phần dữ liệu đầu vào: Hệ thống sử dụng bộ dữ liệu Grocery Inventory được tải từ nền tảng Kaggle thông qua thư viện kagglehub. Dữ liệu bao gồm các thông tin về mã sản phẩm (Product_ID), tên sản phẩm (Product_Name), ngày hết hạn (Expiration_Date) và số lượng tồn kho (Stock_Quantity). Đây là cơ sở để hệ thống xác định danh sách hàng hóa và vị trí lưu trữ phục vụ quá trình mô phỏng.

(2) Thành phần xử lý dữ liệu: Python được sử dụng để đọc và xử lý dữ liệu thông qua các thư viện Pandas và NumPy. Quá trình này bao gồm kiểm tra các trường dữ liệu bắt buộc, chuyển đổi định dạng ngày hết hạn, loại bỏ những bản ghi không hợp lệ và xác định ngày vận hành mô phỏng. Sau đó, hệ thống tính số ngày còn hạn sử dụng của từng sản phẩm và phân loại hàng hóa theo nguyên tắc FEFO thành ba nhóm: hàng sắp hết hạn (màu đỏ), hàng có thời hạn trung bình (màu vàng) và hàng còn thời hạn dài (màu xanh). Dữ liệu sau xử lý được chuyển đổi sang định dạng JSON để JavaScript có thể tiếp nhận và sử dụng trong môi trường mô phỏng.

(3) Thành phần xử lý thuật toán và điều phối robot: Đây là bộ phận đảm nhiệm các chức năng tính toán và điều khiển hoạt động của đội robot AGV. Hệ thống sử dụng thuật toán A* để tìm kiếm tuyến đường trên bản đồ kho dạng lưới, kết hợp cơ chế tính thêm chi phí tại những khu vực có robot khác nhằm hạn chế nguy cơ ùn tắc. Bên cạnh đó, mô hình tích hợp cơ chế CPA để dự đoán nguy cơ va chạm, kiểm tra khoảng cách an toàn và điều chỉnh tốc độ dựa trên khoảng cách phanh. Khi tuyến đường hiện tại không còn phù hợp, robot có thể thực hiện tìm đường lại hoặc đánh giá hướng di chuyển thay thế. Hoạt động của năm robot được quản lý thông qua cơ chế hàng đợi nhiệm vụ và máy trạng thái (Finite State Machine), giúp kiểm soát quá trình lấy hàng, giao hàng, quay về và sạc pin.

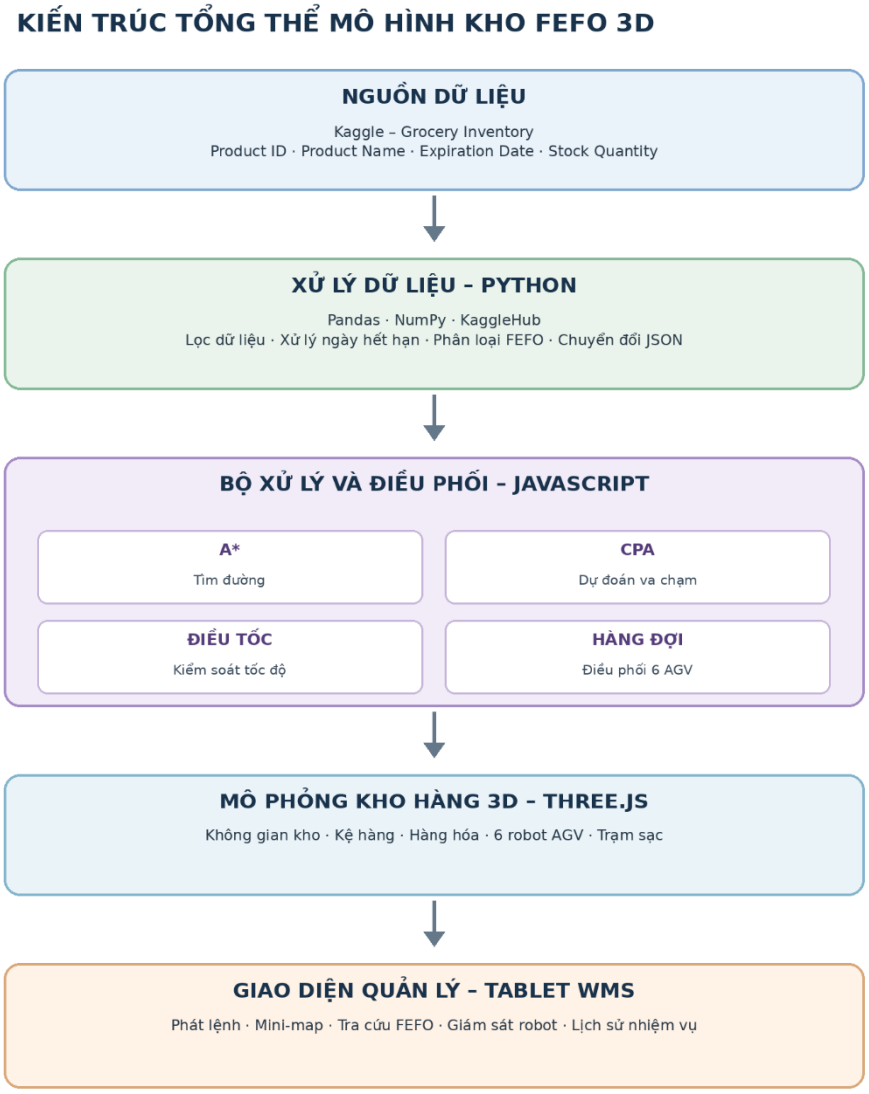
(4) Thành phần mô phỏng và hiển thị 3D: Thư viện Three.js được sử dụng để xây dựng không gian kho hàng dưới dạng mô hình ba chiều. Thành phần này thể hiện các khu vực chức năng, hệ thống kệ hàng, hàng hóa, trạm sạc và năm robot AGV. Thông qua quá trình cập nhật liên tục trong vòng lặp mô phỏng, vị trí, hướng di chuyển, tốc độ và trạng thái của robot được thay đổi tương ứng với kết quả xử lý thuật toán. Ngoài ra, hệ thống hỗ trợ các chế độ quan sát khác nhau, bao gồm góc nhìn quản lý kho 3D và góc nhìn thứ nhất (FPS – First-Person View).

(5) Thành phần giao diện quản lý và giám sát: Giao diện Tablet WMS được xây dựng nhằm hỗ trợ người dùng tương tác với hệ thống. Người dùng có thể tra cứu hàng hóa, lựa chọn robot thực hiện nhiệm vụ, xác định điểm giao hàng, thêm các đơn vào hàng đợi và phát lệnh vận hành. Bên cạnh đó, giao diện còn cung cấp bản đồ thu nhỏ (Minimap), bảng thống kê FEFO, thông tin trạng thái robot và lịch sử lấy hàng. Những thông tin này giúp người dùng theo dõi quá trình vận hành và có cơ sở đánh giá hiệu quả hoạt động của mô hình.

Quy trình hoạt động tổng thể: Dữ liệu hàng hóa từ Kaggle được Python tiếp nhận và xử lý, sau đó chuyển sang JavaScript dưới dạng JSON. JavaScript sử dụng dữ liệu này để khởi tạo môi trường kho

3D, xây dựng bản đồ di chuyển và thiết lập đội robot AGV. Khi người dùng phát lệnh, hệ thống tiến hành phân công nhiệm vụ, tính toán tuyến đường và kiểm tra các điều kiện an toàn. Robot thực hiện quá trình lấy hàng, vận chuyển đến điểm giao, sau đó tiếp tục xử lý các nhiệm vụ còn lại trong hàng đợi. Khi hoàn thành toàn bộ nhiệm vụ, robot tự động quay về trạm sạc. Trong suốt quá trình này, hệ thống liên tục cập nhật trạng thái robot, ghi nhận lịch sử và hiển thị thông tin trên giao diện quản lý.

Sơ đồ kiến trúc tổng thể của mô hình:



Hình 3.1. Sơ đồ kiến trúc tổng thể mô hình kho FEFO 3D với sáu robot AGV.
Nguồn: Nhóm 1 tổng hợp từ mã nguồn mô hình V21.

Hình 3.1. Sơ đồ kiến trúc tổng thể mô hình kho FEFO 3D với năm robot AGV.

(Nguồn: Nhóm 1 tổng hợp từ mã nguồn mô hình)

Như vậy, kiến trúc mô hình thể hiện sự kết hợp giữa xử lý dữ liệu, thuật toán điều phối và công nghệ mô phỏng trực quan. Python đảm nhiệm công việc chuẩn bị dữ liệu đầu vào, JavaScript thực hiện các logic điều khiển và xử lý thuật toán, trong khi Three.js đảm nhiệm việc xây dựng không gian

mô phỏng ba chiều. Sự kết hợp này tạo thành một quy trình vận hành thống nhất, hỗ trợ nhóm nghiên cứu hoạt động của nhiều robot AGV trong môi trường kho hàng thông minh.

3.3. Cài đặt thư viện và công cụ

Nhóm sử dụng Google Colab làm môi trường phát triển và chạy chương trình mô phỏng. Trong đó, Python đảm nhiệm việc tải và xử lý dữ liệu hàng hóa; JavaScript thực hiện thuật toán tìm đường, điều khiển robot và xử lý xung đột. Thư viện Three.js được sử dụng để xây dựng và hiển thị mô hình kho 3D.

Các công cụ chính gồm kagglehub để tải dữ liệu từ Kaggle, pandas và NumPy để xử lý dữ liệu, json để chuyển dữ liệu sang JavaScript. Bên cạnh đó, HTML, CSS, Tailwind CSS và Font Awesome hỗ trợ thiết kế giao diện mô phỏng và bảng điều khiển robot.

Trước khi chạy chương trình, nhóm cài đặt thư viện kagglehub và khai báo các thư viện cần thiết trong môi trường Colab.

3.4. Thiết kế và tiền xử lý dữ liệu

3.4.1. Nguồn dữ liệu Grocery Inventory

Nhóm sử dụng bộ dữ liệu Grocery Inventory từ Kaggle, được tải thông qua thư viện kagglehub. Chương trình tìm và đọc tệp CSV bằng pandas, sau đó kiểm tra các trường dữ liệu bắt buộc gồm Product_ID, Product_Name, Expiration_Date và Stock_Quantity.

Dữ liệu này cung cấp thông tin về mã sản phẩm, tên sản phẩm, số lượng tồn kho và ngày hết hạn, phục vụ việc quản lý và bố trí hàng hóa trong kho mô phỏng.

3.4.2. Tiền xử lý dữ liệu và phân loại FEFO

Sau khi đọc dữ liệu, chương trình chuẩn hóa định dạng ngày hết hạn, kiểm tra và loại bỏ các bản ghi thiếu mã hoặc tên sản phẩm, có ngày hết hạn không hợp lệ, số lượng tồn kho không hợp lệ hoặc đã hết hạn so với ngày mô phỏng.

Ngày vận hành được chương trình lựa chọn tự động dựa trên dữ liệu hạn sử dụng nhằm ưu tiên có đủ ba nhóm FEFO. Các bản ghi hợp lệ được tính số ngày còn hạn và sắp xếp theo thứ tự tăng dần.

Hàng hóa được phân loại thành ba nhóm theo nguyên tắc FEFO (First Expired, First Out): đỏ (0–15 ngày), vàng (16–45 ngày) và xanh (trên 45 ngày). Kết quả xử lý sau đó được chuyển sang JSON để sử dụng trong mô hình 3D, hỗ trợ nhận diện và ưu tiên lựa chọn hàng gần hết hạn.

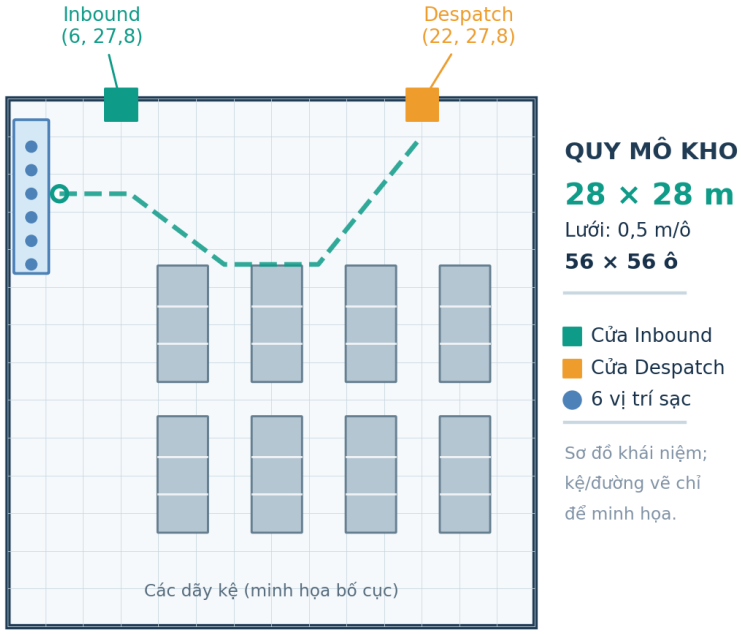
3.5. Xây dựng mô hình tối ưu tuyến đường

3.5.1. Mục tiêu và tổ chức vận hành

Trong kho thông minh nhiều AGV, đường ngắn nhất theo hình học chưa chắc là đường hiệu quả nhất do chờ tại giao lộ hoặc tránh robot khác. V21 dùng A* trên lưới có chi phí tránh vật cản động, kết hợp kiểm soát chuyển động và phục hồi khi bị chặn. Chu trình: PICK → DROP → (đơn tiếp theo nếu có) → RETURN → DOCK_TURN → CHARGE → IDLE.

3.5.2. Không gian kho và mô hình lưới

Mặt bằng mô phỏng có kích thước 28 × 28 m; bước lưới 0,5 m tạo 56 × 56 = 3.136 ô. Các ô kệ bị đánh dấu không đi được; thuật toán mở rộng theo bốn hướng (trái, phải, trước, sau).



Hình – Mặt bằng kho 28 × 28 m, khu Inbound/Despatch và sáu ô sạc; hình khái niệm từ tài liệu gốc.

Trên lưới bốn hướng, khoảng cách Manhattan phù hợp làm heuristic; khoảng cách Euclid dùng để đo độ gần giữa các robot.

$d_M = |x_2 - x_1| + |z_2 - z_1|$; $d_E = \sqrt{[(x_2 - x_1)^2 + (z_2 - z_1)^2]}$

Ví dụ A(4;6) → B(11;9): Manhattan = 10 bước = 5,0000 m; Euclid = $\sqrt{58} \times 0{,}5 = 3{,}8079$ m. Như vậy độ dài tối thiểu trên lưới bốn hướng có thể lớn hơn đường thẳng.

const h = n => Math.abs(n.x-goal.x)+Math.abs(n.z-goal.z); const directions = [[1,0],[-1,0],[0,1],[0,-1]];

3.5.3. Dijkstra, A và cơ chế tránh robot*

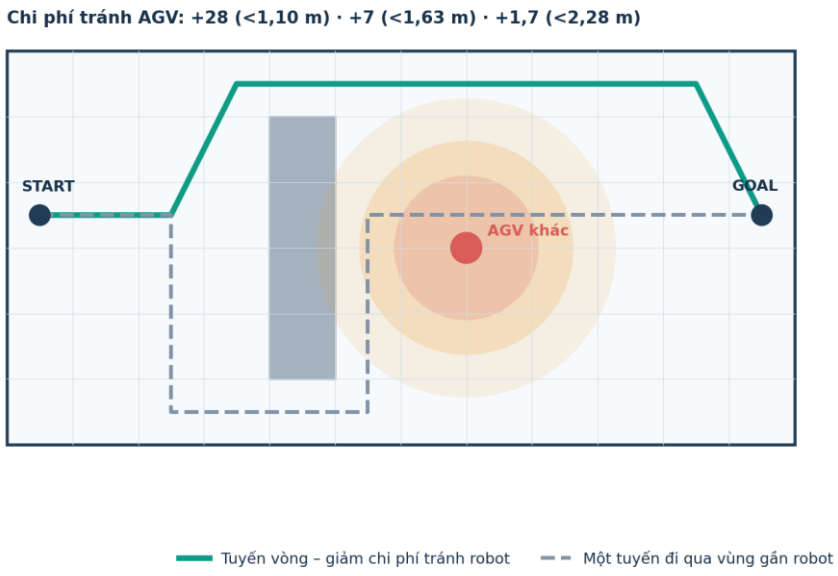
Dijkstra cập nhật d(v)=min[d(v), d(u)+w(u,v)] với trọng số không âm. V21 sử dụng A* làm bộ lập tuyến chính: g(n) là chi phí đã đi, h(n) là ước lượng Manhattan và f(n) là độ ưu tiên chọn nút. Dijkstra chỉ được dùng làm cơ sở lý thuyết đối chiếu, không phải bộ lập tuyến chính đang chạy.

$f(n) = g(n) + h(n)$; $h(n) = |x_n - x_goal| + |z_n - z_goal|$

gScore[sourceKey]=0; push({n:start,f:h(start)}); if(!gridMap[nx][nz].walkable) continue; // Ô kệ bị loại

Chi phí A* nền bằng 1 cho mỗi bước, có thể cộng chi phí mềm để tránh khu vực gần robot khác. Các mức cộng không phải mét hoặc giây; không có việc khóa tuyệt đối mọi ô đang gần robot.

Khoảng cách đến robot khác	Chi phí cộng
d < 1,10 m	+28
1,10 ≤ d < 1,63 m	+7
1,63 ≤ d < 2,28 m	+1,7
d ≥ 2,28 m	0



Hình – Minh họa A* ưu tiên tuyến vòng có chi phí thấp hơn khi gần AGV khác.

if(d<SAFE_GAP+.12) cost+=28; else if(d<SAFE_GAP+.65) cost+=7; else if(d<SAFE_GAP+1.3) cost+=1.7;

Ví dụ minh họa: tuyến 12 bước qua một ô phạt 28 có tổng chi phí 40, còn tuyến 16 bước không phạt có chi phí 16. A* có thể chọn đường dài hơn 2 m nhưng ít đi vào vùng bất lợi hơn. Ngoài ra, robot di chuyển có thể tạo phụ phí +8 ở điểm dự báo sau 0,65 s.

3.5.4. Chu trình PICK DROP RETURN và quy tắc điều hướng

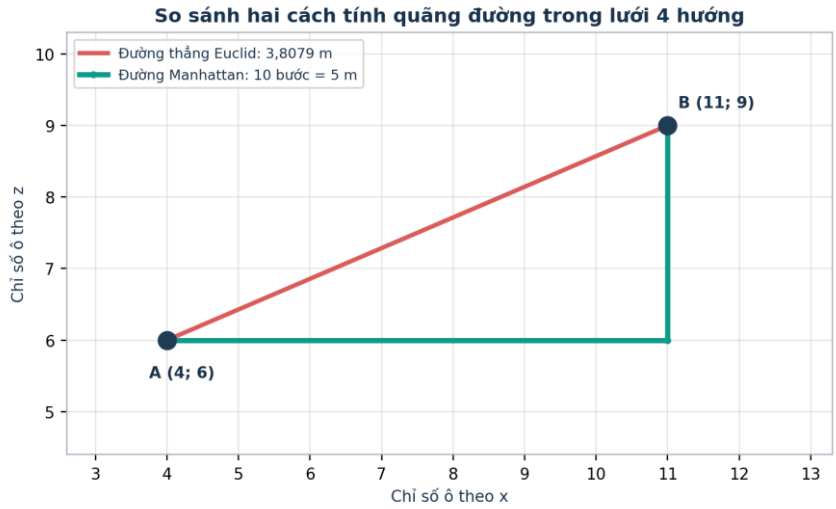
Chiều dài hành trình có thể tính theo D = D_PICK + D_DROP + D_RETURN. Tuy nhiên lịch sử thời gian nhiệm vụ chỉ ghi đến khi giao hàng xong; chặng RETURN chỉ phát sinh khi không có đơn chờ. Thời gian lý tưởng T = D/v_max không kể tăng giảm tốc, quay hướng và nhường đường.

D_total = D_PICK + D_DROP + D_RETURN T_ideal = D / 2,3

Khi bị chặn, robot giảm tốc/chờ, thử tính đường lại có nhịp tối thiểu 2,6 s, sau đó xét vòng tránh. Bảng dưới giữ nguyên sáu quy tắc tác động đến giai đoạn lập tuyến và bố trí điểm đích.

QT	Quy tắc / Điều kiện – xử lý	Code minh họa V21
01	Tránh kệ: bỏ ô walkable=false; tạo đệm vật cản tĩnh 0,25 m.	if(!gridMap[nx][nz].walkable) continue;
02	Tránh robot: cộng +28 / +7 / +1,7 theo ba vùng khoảng cách.	if(d<SAFE_GAP+.12) cost+=28;
03	Dự báo robot khác sau 0,65 s; nếu gần điểm xét thì cộng +8.	if(other.speed>.1 && d<2.5) {... cost+=8;}
04	Cấm đường tắt phía sau ô sạc trong A* thông thường.	if(world.x<2.22 && world.z>17.55 && world.z<26.85) continue;
05	Đường RETURN bị cộng phí gần cửa: +85 gần cửa, +6 vùng mở rộng.	if(returnTrip) cost+=returnDoorPenalty(world.x,world.z);
06	Điểm giao mặc định theo robot lệch nhau 1,40 m theo X.	x:dock.x+(r.id-2.5)*1.40

Về phạm vi tối ưu:



Hình – So sánh Euclid và Manhattan trên lưới bốn hướng (ví dụ A→B).

Chu trình trạng thái và điểm kết thúc phép đo

Trạng thái	Xử lý chính
PICK	Lập đường đến ô hàng và xác nhận lấy kiện.
DROP	Chở kiện tới cửa hoặc điểm giao; kết thúc nhiệm vụ.
RETURN	Chỉ về sạc khi robot đã hết đơn chờ.
DOCK_TURN → CHARGE → IDLE	Vào ô, quay 180°, nạp pin và sẵn sàng.

A* giải quyết lập đường trên đồ thị; vùng phạt chủ yếu dựa vào vị trí hiện tại hoặc dự báo ngắn. Do chưa có hệ thống đặt chỗ từng ô theo thời gian, hai tuyến A* riêng biệt vẫn có thể xung đột tại giao lộ. Vì vậy mô hình phải bổ sung CPA, điều tốc và kiểm tra bước đi ở mục 3.6.

3.6. Kiểm soát xung đột và va chạm

3.6.1. Kiến trúc nhiều lớp và dự đoán CPA

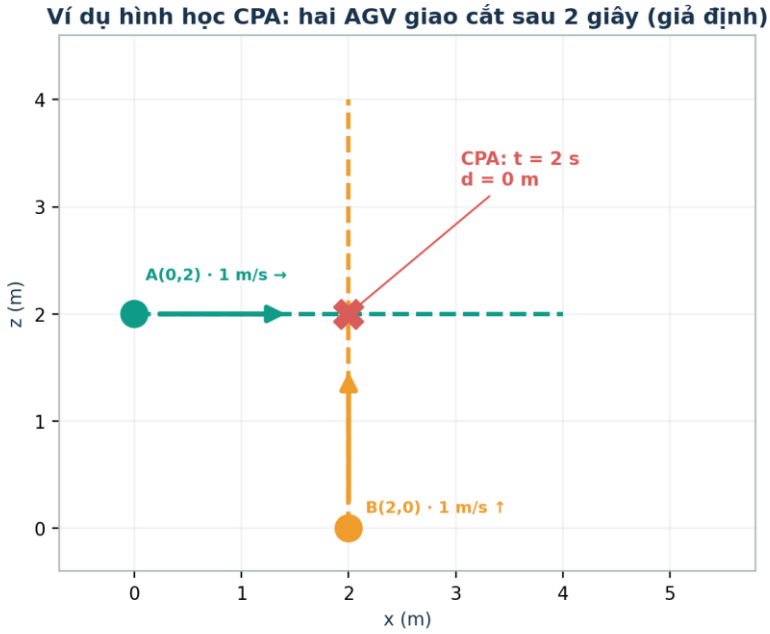
V21 dùng cảm biến ảo dựa trên tọa độ, không phải LiDAR thực. Các lớp kiểm soát gồm: đo Euclid → CPA dự báo xung đột → giảm tốc theo phanh → f5AllowedMove() xác nhận bước đi → lập lại đường/vòng tránh khi bị chặn. Một bước hợp lệ tại thời điểm lập tuyến chưa bảo đảm mọi robot sẽ luôn di chuyển không xung đột.

$$d_AB = \sqrt{[(x_B - x_A)^2 + (z_B - z_A)^2]}$$

CPA (Closest Point of Approach) giả định hai robot giữ vận tốc tương đối trong cửa sổ T = 2,4 s. Từ r = p_B−p_A và v_r = v_B−v_A, chương trình tìm thời điểm hai robot gần nhau nhất:

$$t_CPA = clamp[-(r \cdot v_r) / |v_r|^2, 0, T]$$

$$d_CPA = |r + v_r\, t_CPA| \ (T = 2{,}4\ s)$$



Hình – Ví dụ hình học CPA: hai robot chuyển động vuông góc và giao cắt sau 2 giây.

Ví dụ: A(0;2) đi sang phải 1 m/s; B(2;0) đi lên 1 m/s. Khoảng cách ban đầu 2,8284 m nhưng t_CPA = 2 s và d_CPA = 0 m. Vì vậy chỉ dùng khoảng cách hiện tại sẽ bỏ sót xung đột tương lai.

const t=rv2>1e-8 ? Math.max(0,Math.min(CONFLICT_HORIZON, -(rx*rvx+rz*rvz)/rv2)) : 0; const closest=Math.hypot(rx+rvx*t,rz+rvz*t);

Khi d_CPA < SAFE_GAP + 0,16 = 1,14 m và các điều kiện hướng chuyển động được thỏa mãn, chương trình hạn chế vận tốc; đây là dự đoán ngắn hạn, cần cập nhật theo từng chu kỳ.

3.6.2. Điều tốc theo phanh và xác nhận bước di chuyển

Thông số điều khiển: SAFE_GAP = 0,98 m; MAX_SPEED = 2,3 m/s; ACCEL = 1,15 m/s²; BRAKE = 2,7 m/s²; SENSOR_RANGE = 4,8 m. Tốc độ an toàn tính theo quãng đường còn trống với mô hình phanh động học:

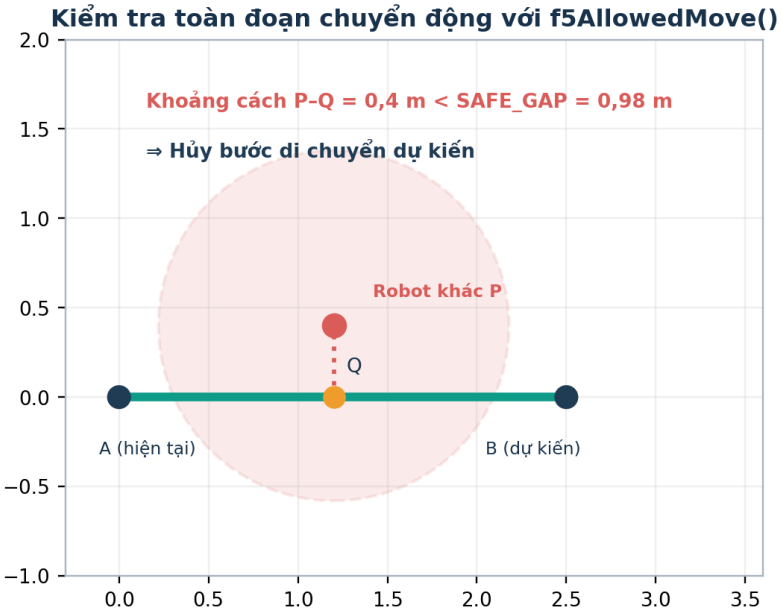
$d_brake = v^2 / (2 \times BRAKE)$ $v_safe = \sqrt{(2 \times BRAKE \times clearance)}$

Tại tốc độ 2,3 m/s, quãng đường phanh lý tưởng là 0,9796 m. Khi khoảng trống d_free = 0,50 m, vận tốc giới hạn là $\sqrt{(2\times2{,}7\times0{,}5)} = 1{,}6432$ m/s (trước khi xét các điều kiện khác).

Trong f5PredictiveControl(), forward là hình chiếu khoảng cách theo hướng tiến; lateral là độ lệch ngang. Vùng an toàn hình tròn có R = SAFE_GAP + 0,08 = 1,06 m; longitudinal = $\sqrt{(R^2 - lateral^2)}$. Chương trình tiếp tục trừ biên tiến 0,04 m:

$clearance = \max(0, forward - \sqrt{(1{,}06^2 - lateral^2)} - 0{,}04)$

const longitudinal=Math.sqrt(Math.max(0, (SAFE_GAP+.08)**2-lateral*lateral)); const clearance=Math.max(0,forward-longitudinal-.04); const cap=Math.sqrt(2*BRAKE*clearance);



Hình – Kiểm tra khoảng cách nhỏ nhất từ robot khác đến toàn đoạn bước đi dự kiến.

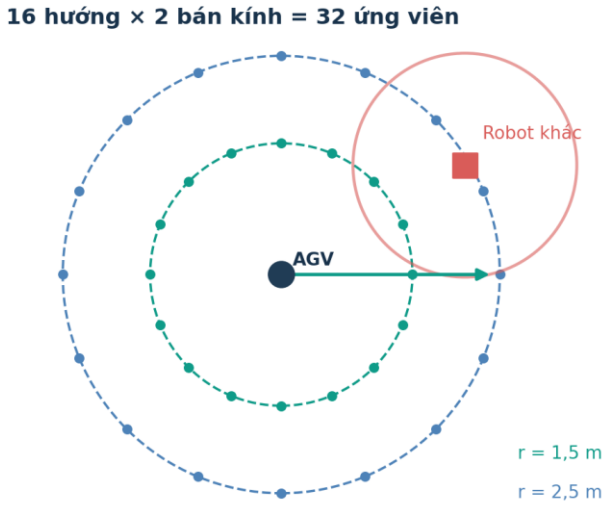
f5AllowedMove() là lớp cuối: tính hình chiếu Q của vị trí robot khác P lên đoạn dự định A→B. Nếu |P−Q| < 0,98 m thì bước đó bị hủy và robot dừng/nhường. Cơ chế này so với vị trí robot khác tại thời điểm kiểm tra, chưa phải kiểm chứng quỹ đạo liên tục đồng thời.

u = clamp[(P−A)·(B−A) / |B−A|², 0, 1]; Q=A+u(B−A)

if(f5Dist(closest,other)<gap) return false; if(step>0 && !f5AllowedMove(a,next,r.id)){speed=0;step=0;}

3.6.3. Phục hồi, quét 360° và thứ tự cập nhật

Khi không tiến được, robot ưu tiên thử tìm đường mới (REPLAN_INTERVAL = 2,6 s). Nếu tiếp tục bị chặn, robot dò điểm trung gian theo ba bán kính 1,5/2,5/3,5 m và tám hướng. Trường hợp không có tuyến đủ lâu (ít nhất 14 s), hết thời gian nghỉ quét và không có robot khác đang đi qua gần đó, chương trình mới cân nhắc xoay 360°.



Hình – Bộ ứng viên sau quét: 16 hướng × 2 bán kính (1,5 và 2,5 m) = 32 điểm.

Khi đánh giá 32 điểm, chương trình loại vị trí quá gần robot khác ở hiện tại hoặc các thời điểm dự báo 0,8 s và 1,5 s. Quét mô phỏng quay 3,2 rad/s và có cooldown 25 s. Việc quét 360° chỉ ở nhánh "không có tuyến", không áp dụng cho mọi trường hợp tạm dừng.

if(waited>=BLOCKED_SCAN_AFTER && fleet5Clock>=r.nextScanAllowedAt){ if(!r.path.length) f5StartScan(r); }

Thứ tự cập nhật năm robot dịch chỉ số đầu tiên theo thời gian để hạn chế robot luôn được xét trước. Đây KHÔNG phải quyền ưu tiên chính thức tại giao lộ.

priority(t) = floor(t / 4) mod 6; chu kỳ lặp = 6 × 4 = 24 s

3.6.4. Quy tắc cảm biến và điều tốc

QT	Điều kiện / hành động	Code minh họa V21
07	Robot phía trước, lệch ngang <0,8036 m: giới hạn tốc độ phanh	forward>.12 && lateral<SAFE_GAP*.82
08	Robot bên/sau trong phạm vi 1,8 m: giảm tốc thận trọng.	if(dist<SIDE_RANGE) limit=Math.min(limit,cap);
09	CPA dự đoán d_CPA <1,14 m: hạn chế vận tốc.	if(closest<SAFE_GAP+.16){risk +;...}
10	Robot trước: v_safe=√(2·BRAKE·free).	Math.sqrt(2*BRAKE*free)

QT	Điều kiện / hành động	Code minh họa V21
11	Gần kệ hoặc góc kệ: dò forwardGap và giảm tốc.	allowed=Math.min(allowed,Mat max(.25,vSafe));
12	Trước/sau cua: hạ tốc theo độ cong và góc cần xoay.	targetSpeed=Math.min(targetSp ,safeTurnSpeed);
13	Lệch hướng >1,2 rad: xoay tại chỗ; >0,4 rad: tốc ≤0,51 m/s.	if(steeringInPlace) targetSpeed=(

3.6.5. Quy tắc phòng tránh và phục hồi

QT	Điều kiện / hành động	Code minh họa V21
14	Nếu bước tiến đến gần robot khác dưới SAFE_GAP thì hủy bước.	!f5AllowedMove(a,next,r.id)
15	Ghi một xung đột khi trạng thái chuyển từ không bị chặn sang bị chặn.	if(blockedByRobot&&!r.blockedNow) conflictCount++;
16	Thử lập lại đường, giới hạn chu kỳ 2,6 s.	fleet5Clock-r.lastRouteTryAt<REPLAN_INTERVAL
17	Rẽ vòng: 3 bán kính × 8 hướng, chọn đường phù hợp sau khi lọc.	for(const radius of [1.5,2.5,3.5]) {...}
18	Robot khác đang đi qua gần 2,5 m: nhường, không quét.	if(nearbyMoving){r.state='NHƯỜNG...';return false;}
19	Không có tuyến ≥14 s và đủ cooldown mới xét quét 360°.	if(!r.path.length) f5StartScan(r);
20	Loại điểm quét quá gần robot dự báo ở t=0; 0,8; 1,5 s.	[0,.8,1.5].some(t=> ... <SAFE_GAP+.30)
21	Không quét lặp liên tiếp, cách nhau ≥25 s.	r.nextScanAllowedAt=fleet5Clock+SCAN_COOLDOWN;
22	Luân phiên robot cập nhật trước, đổi mỗi 4 s.	Math.floor(fleet5Clock/4)%fleet5.length

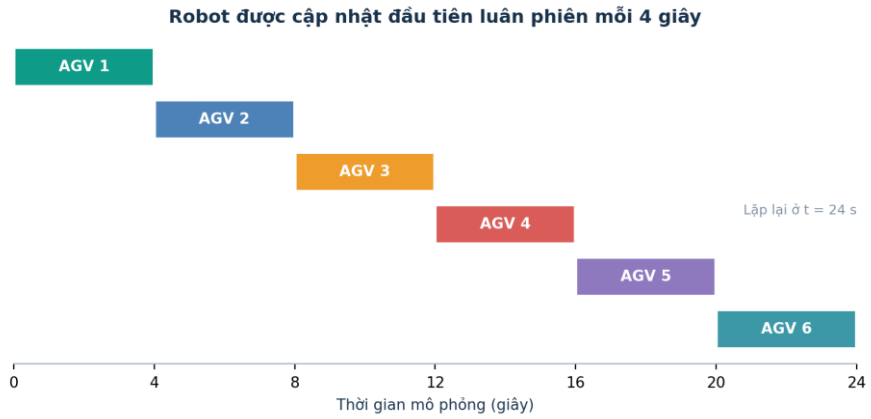
3.6.6. Ưu điểm và giới hạn kiểm soát an toàn

• Ưu điểm: kết hợp A* có vùng phạt, CPA, điều tốc phanh, kiểm tra bước, tìm lại tuyến và thử vòng tránh; việc phân lớp giúp giảm nguy cơ tiến vào vùng gần robot khác.

• Giới hạn: chưa có đặt chỗ theo thời gian (MAPF), chưa bảo đảm không bế tắc ở mọi bố cục; cập nhật từng robot không tương đương mô phỏng va chạm liên tục đồng thời.

• Việc ghi collisionCount bằng 0 trong lịch sử không tự chứng minh không va chạm nếu chưa có bộ phát hiện va chạm độc lập; dữ liệu cảm biến là mô phỏng, không phải đo LiDAR vật lý.

Định hướng kỹ thuật: dùng Cooperative A*/CBS hoặc đặt chỗ giao lộ, đo khoảng cách liên tục giữa mọi cặp robot và lưu nhật ký sự kiện chặn–thoát chặn cho Chương 5.

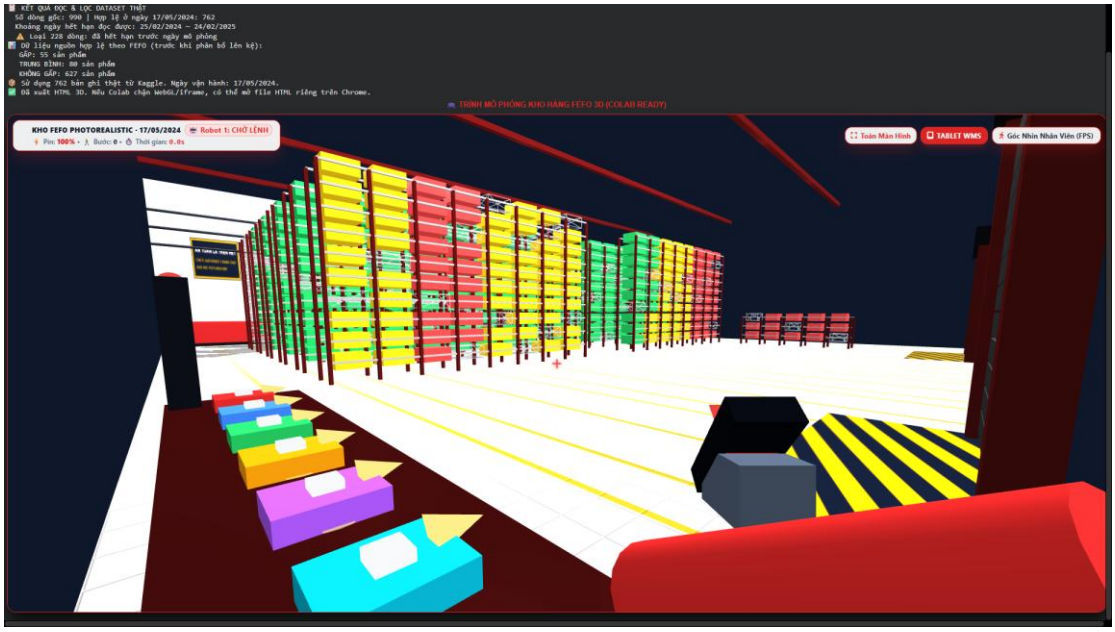


Hình – Luân phiên robot được cập nhật đầu tiên theo chu kỳ bốn giây.

3.7. Giao diện mô phỏng và giám sát

3.7.1. Công nghệ, không gian 3D và camera

Python chuẩn bị dữ liệu đầu vào; JavaScript điều khiển mô phỏng; Three.js dựng Scene, Camera, Renderer, robot, mặt sàn 28 × 28 m và các kệ. HTML/CSS tạo lớp điều khiển; Colab hiển thị mô phỏng qua trình duyệt. Hai góc quan sát chính: FPS để di chuyển trong kho và góc quản lý để theo dõi đội robot.



Hình – Ảnh giao diện kho user

Camera PerspectiveCamera tạo phối cảnh thông qua fov (góc nhìn dọc), aspect (tỷ lệ khung hình), near/far (mặt phẳng cắt). Công thức hình học mô tả góc nhìn ngang:

$\theta_{horizontal} = 2 \arctan[\text{aspect} \times \tan(\theta_{vertical} / 2)]$

3.7.2. Tương tác FPS và Mini-map

Thành phần	Cách hoạt động / phép tính quan trọng
Bàn phím W/A/S/D	Tạo vectơ di chuyển theo camera; chuẩn hóa đường chéo. $\Delta s = v \cdot \Delta t$ nên không phụ thuộc FPS.
Chuột	Thay đổi yaw và pitch theo $\Delta x, \Delta y$; có thể giới hạn pitch để tránh lật camera.
Raycasting	Chuyển pixel sang NDC: $x = 2x_{mouse}/W - 1$, $y = 1 - 2y_{mouse}/H$; dùng Raycaster chọn mesh.
Mini-map 2D	Ánh xạ x,z trong thế giới thành pixel; cần thống nhất hệ tọa độ với bản đồ kho.

Ví dụ tại 60 FPS, vận tốc camera 2 m/s tạo bước lý tưởng 0,0333 m/khung. Đây là phép tính minh họa. Mini-map chỉ thể hiện vị trí/tuyến; hiệu suất hiển thị còn phụ thuộc thiết bị.

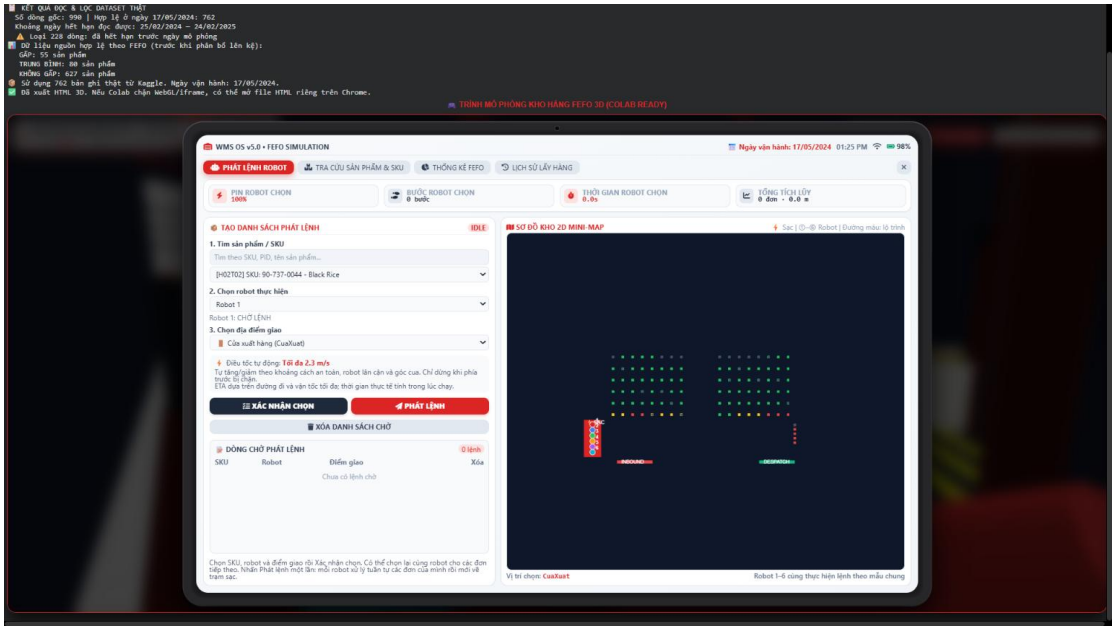
Ví dụ quy đổi tương tác

Nhấp chuột tại pixel (960;360) trong khung 1280×720 đưa về NDC=(0,5;0), sau đó Raycaster bắn tia để xác định mesh được chọn. Với Mini-map 280×280 px giả định tọa độ kho từ −14 đến +14, robot ở (−7;4) m tương ứng (70;180) pixel. Nếu V21 dùng gốc tọa độ khác, cần điều chỉnh phép quy đổi trước khi sử dụng.

```
const xNDC = 2*mouseX/width-1; const yNDC = 1-2*mouseY/height; raycaster.setFromCamera({x:xNDC,y:yNDC},camera);
```

3.7.3. Tablet WMS, hàng đợi và ô sạc

Tablet WMS cho phép tìm theo SKU/Product ID/tên/vị trí, chọn robot 1–6, điểm giao, lưu lệnh chờ và phát lệnh theo nhóm. Hệ thống phân biệt pendingDispatches (chưa phát), queuedSlotIds (ô hàng đang xếp hàng) và reservedSlotIds (ô đang được robot giữ chỗ); mỗi robot có jobQueue riêng.



Hình – Ảnh Tablet WMS/Mini-map trong mô hình từ tài liệu được cung cấp.

QT	Quy tắc điều phối	Code minh họa V21
23	Ra ô sạc qua điểm tiếp cận x=2,75; RETURN tiến thẳng vào ô rồi mới xoay 180°.	path.push({x:DOCK_APPROACH_X,z:r.bay.z});
24	Giữ chỗ tránh trùng SKU theo ô; làm hết jobQueue rồi mới gọi RETURN.	queuedSlotIds.add(cmd.slotId); fleetReturn(r);

Ví dụ: robot 1 có ba nhiệm vụ A→C→F sẽ lấy đơn tiếp theo ngay từ vị trí giao trước đó, sau đơn F mới về trạm. Quy tắc này hạn chế các lượt RETURN trung gian nhưng cần đo mới xác định được thời gian tiết kiệm.

Quy trình thao tác cho người vận hành

• Tìm hàng và kiểm tra thông tin FEFO → chọn robot và điểm giao → đưa vào hàng đợi → phát lệnh.

• Quan sát 3D, Mini-map, trạng thái robot, mức pin và danh sách nhiệm vụ; xuất CSV khi cần đánh giá.

Dữ liệu hiển thị và ý nghĩa theo dõi

Thông tin	Mục đích giám sát
Robot/phase/state	Xác định robot đang lấy hàng, giao hàng, chờ, về trạm hay sạc.

Thông tin	Mục đích giám sát
Speed, battery, distance	Theo dõi tốc độ, pin mô phỏng và quãng đường tích lũy.
Pending / jobQueue	Phân biệt lệnh chưa phát và nhiệm vụ đang chờ từng robot.
Task history	Thu thập SKU, thời gian, quãng đường, xung đột để xuất CSV.

3.7.4. Vòng lặp cập nhật, đo thời gian và đánh giá FPS

Trong animate(), mỗi khung nhận Δt, cập nhật chuyển động bằng updateFiveFleet(Δt) rồi dựng hình. Khoảng di chuyển lý tưởng mỗi bước là Δs=vΔt, thực tế còn phụ thuộc điều tốc và việc từ chối bước không an toàn.

function updateFiveFleet(delta){ fleet5Clock+=delta; const priority=Math.floor(fleet5Clock/4)%Math.max(1,fleet5.length); for(let k=0;k<fleet5.length;k++) f5Advance(fleet5[(k+priority)%fleet5.length],delta); }

V21 lưu các trường: robot, SKU, estimated, actual, meters, conflicts, collisions và hỗ trợ xuất CSV. ETA lý tưởng dựa trên độ dài đường và tốc độ tối đa; actual tính từ khi nhiệm vụ bắt đầu thực hiện đến lúc DROP hoàn tất, không gồm RETURN sau đó.

T_estimated = D_plan / v_max ; T_actual = t_finish − t_start ; ΔT = T_actual − T_estimated

Ví dụ minh họa: D_plan = 23 m, v_max=2,3 m/s ⇒ T_estimated=10 s. Nếu T_actual=14,5 s thì vượt 4,5 s (45%). Đây là ví dụ tính, không phải kết quả đo thử nghiệm.

Tổng hợp giá trị của mô hình và giới hạn

Đã hỗ trợ	**Cần tiếp tục phát triển**
Kho 3D, FPS, Mini-map, Tablet WMS và hàng đợi sáu robot.	Kiểm soát camera không xuyên kệ, đồng bộ UI ở thiết bị yếu.
A* tránh kệ + phạt robot; CPA; vận tốc phanh; kiểm tra bước.	MAPF có chiếm dụng thời gian, chống bế tắc chéo tại giao lộ.
Đường về trạm sạc, xoay tại bến và xử lý đơn liên tiếp.	Tiêu hao pin thực, tối ưu phân công nhiệm vụ tự động.
Lịch sử tốc độ/nhiệm vụ có thể xuất CSV.	Đo va chạm độc lập, lưu dữ liệu nhiều phiên và thí nghiệm đối chứ

CHƯƠNG 4: QUY TRÌNH VẬN HÀNH

4.1. Thiết lập môi trường vận hành

Mô hình được phát triển trên nền tảng Python kết hợp JavaScript và Three.js.

Các thư viện chính được sử dụng gồm:

- pandas
- numpy
- kagglehub
- json
- glob
- os
- Three.js
- Tailwind CSS
- Font Awesome

Trong đó:

Python dùng để xử lý dữ liệu.

- Three.js dùng để mô phỏng môi trường kho 3D.
- Tailwind CSS hỗ trợ xây dựng giao diện người dùng.
- KaggleHub hỗ trợ truy xuất dữ liệu Inventory từ Kaggle.

4.2. Chuẩn hóa và xử lý dữ liệu

Sau khi tải dữ liệu từ Kaggle, hệ thống thực hiện các bước:

Bước 1: Kiểm tra dữ liệu

Các trường bắt buộc gồm:

- Product_ID
- Product_Name
- Expiration_Date
- Stock_Quantity

Các bản ghi thiếu dữ liệu sẽ bị loại bỏ.

Bước 2: Chuyển đổi định dạng ngày tháng

Ngày hết hạn được chuyển sang dạng DateTime để thuận tiện cho việc tính toán.

Bước 3: Tính số ngày còn hạn sử dụng

Số ngày còn hạn được xác định bằng:

$D_i = E_i - T$

Trong đó:

- Di: số ngày còn hạn.
- Ei: ngày hết hạn.
- T: ngày vận hành mô phỏng.

Bước 4: Phân loại FEFO

Hệ thống chia sản phẩm thành ba nhóm:

Nhóm	Thời hạn
Đỏ	0-15 ngày
Vàng	16-45 ngày
Xanh	Trên 45 ngày

Những hàng hóa thuộc nhóm đỏ được ưu tiên xử lý trước nhằm giảm thiểu nguy cơ hết hạn.

4.3. Thứ tự thực thi chống va chạm

Sau khi A* tạo tuyến, bộ điều khiển thực hiện các bước theo cùng một thứ tự ở mỗi lần cập nhật. Robot lấy waypoint tiếp theo, tính khoảng cách và hướng di chuyển, sau đó áp dụng giới hạn tốc độ theo vật cản tĩnh, robot lân cận và khoảng cách phanh.

Bước 1. Dự đoán xung đột bằng CPA trong cửa sổ 2,4 giây.

Bước 2. Giảm tốc hoặc chờ nếu khoảng trống phía trước không đủ để phanh an toàn.

Bước 3. Kiểm tra toàn bộ đoạn quét của bước di chuyển bằng f5AllowedMove(); nếu khoảng cách nhỏ hơn SAFE_GAP = 0,98 m thì từ chối bước.

Bước 4. Nếu robot bị chặn đủ lâu, thử lập tuyến lại sau 2,6 giây, tiếp đó thử cooperative detour; scan 360° chỉ được kích hoạt sau 14 giây và có cooldown 25 giây.

Bước 5. Ghi sự kiện vào log, cập nhật trạng thái WMS và tiếp tục chu trình PICK → DROP → RETURN → DOCK_TURN → CHARGE.

CHƯƠNG 5: THÍ NGHIỆM VÀ ĐÁNH GIÁ

5.1. Thiết lập thí nghiệm

Mục tiêu là đánh giá khả năng hoàn thành đơn, sai lệch thời gian, điều tốc–nhường đường giữa nhiều robot và khả năng phục hồi, xử lý hàng đợi, quay về trạm sạc của mô hình V21.

5.1.1. Cấu hình và phạm vi thử nghiệm

Thông số kỹ thuật	Giá trị	Thông số kỹ thuật	Giá trị
Mặt bằng kho	28 × 28 m	Lưới định tuyến	56 × 56 ô (0,5 m)
Robot AGV	Tối đa 6	Vận tốc lớn nhất	2,3 m/s
Gia tốc / phanh	1,15 / 2,7 m/s²	Khoảng cách an toàn	0,98 m
Cửa sổ CPA	2,4 s	Thử lập lại tuyến	2,6 s
Xét quét khi không tuyến	≥ 14 s	Bước thời gian kiểm thử	0,1 s

5.1.2. Thiết kế bảy kịch bản

	Thiết lập	Mục đích đánh giá
01	1 robot / 1 đơn	Khả năng thực hiện nhiệm vụ cơ sở
02	3 robot / 3 đơn	Vận hành và nhường đường mức tải vừa
03	6 robot / 6 đơn	Vận hành đồng thời cả sáu robot
04	2 robot đối đầu	Phát hiện xung đột tại hành lang z = 21,25 m
05	1 robot mất tuyến	Vách chắn tạm được tháo ở giây 30; kiểm tra phục hồi
06	6 robot / 12 đơn	Hai đơn mỗi robot; kiểm tra jobQueue
07	6 robot về sạc	Cùng sáu đơn TN03; chạy tiếp đến khi về sáu ô sạc

5.1.3. Phương pháp đo và chỉ số

Mã V21 ghi lịch sử nhiệm vụ (ETA, thời gian thực tế, quãng đường, sự kiện bị chặn). Bộ thử nghiệm bổ sung khoảng cách tối thiểu, thời gian dừng, số lần lập lại tuyến, thử vòng tránh, quét 360° và bước chuyển động bị từ chối.

Tỷ lệ hoàn thành $R = N_{\text{hoàn thành}} / N_{\text{tiếp nhận}} \times 100\%$; thời gian trung bình $\bar{T} = \Sigma T_i / N$; quãng đường trung bình $\bar{D} = \Sigma D_i / N$; thông lượng tham khảo $Q = 3600N/T$ kết thúc (đơn/giờ).

5.1.4. Quy ước ghi nhận

Thời gian thực tế tính từ lúc robot bắt đầu xử lý nhiệm vụ đến khi giao xong, không tính thời gian chờ trong jobQueue và chặng RETURN sau giao đơn cuối. ETA là quãng đường dự kiến chia MAX_SPEED; do đó không phải dự báo thời gian vận hành đầy đủ. Tổng quãng đường đội robot có thể lớn hơn tổng quãng đường các đơn vì còn hành trình về trạm.

Sự kiện xung đột được đếm khi robot chuyển sang trạng thái bị robot khác chặn; thời gian chặn/dừng cộng theo robot·giây. Khoảng cách tối thiểu là kết quả lấy mẫu tại mỗi 0,1 s, không phải đo tiếp xúc liên tục. TN03 và TN07 dùng chung sáu đơn nhưng mốc kết thúc khác nhau.

5.2. Đánh giá kết quả thí nghiệm

5.2.1. Tổng hợp kết quả TN01–TN07

N	Đơn xong	Kết thúc (s)	ETA TB (s)	Thực TB (s)	Đường TB (m)	Xung đột	dmin (m)
1	1/	35,00	12,31	35,00	26,63	0	—
2	3/	69,30	17,20	56,13	38,46	6	1,0531
3	6/	99,30	22,65	76,30	54,17	27	1,0012
4	2/	18,50	3,48	12,60	7,91	3	1,0002
5	1/	62,00	12,31	62,00	26,66	0	—
6	2	213,40	24,27	90,77	63,25	56	0,9810
7	6/	164,90	22,65	76,30	54,17	27	0,9804

Bảng 5.1. Dữ liệu gốc từ báo cáo V21. dmin: khoảng cách tâm nhỏ nhất được lấy mẫu; "—": không áp dụng cho một robot.

Hình 5.1. So sánh thời gian dự tính và thực tế trung bình trong 7 kịch bản (headless).

5.2.2. Đánh giá thời gian và năng suất

Cả 7/7 kịch bản hoàn thành số đơn được giao trong lần chạy tương ứng. ETA thấp hơn thời gian thực tế ở tất cả tình huống do ETA lấy đường kế hoạch chia vận tốc tối đa, chưa gồm tăng/giảm tốc, quay hướng, chờ đường và đổi tuyến.

TN01 hoàn tất sau 35,00 s; TN02 sau 69,30 s; TN03 sau 99,30 s. TN06 hoàn tất 12/12 đơn trong 213,40 s với thời gian thực tế trung bình 90,77 s/đơn. Không quy chênh lệch giữa TN01–TN03 hoàn toàn cho số robot vì tuyến và SKU đầu vào khác nhau..

5.2.2.1. Phân tích sai lệch và thông lượng

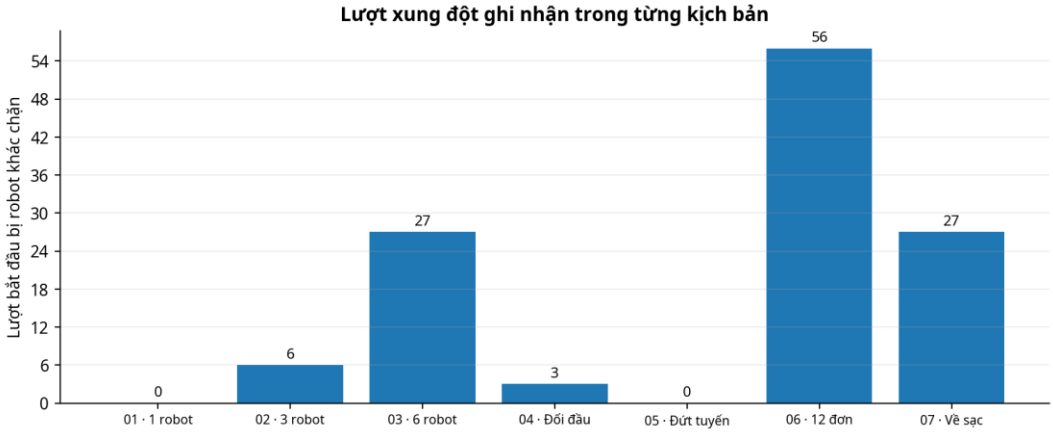
Chênh lệch ETA–thực tế trung bình lần lượt là +22,69 s (TN01), +38,93 s (TN02), +53,65 s (TN03), +9,12 s (TN04), +49,69 s (TN05), +66,51 s (TN06) và +53,65 s (TN07). TN05 tăng thời gian vì có vách chắn tạm; TN06 có quãng đường dài hơn và phát sinh nhiều sự kiện chờ hơn. Không thể tách riêng đóng góp của từng nguyên nhân từ bảng tổng hợp.

Thông lượng tính theo đúng thời điểm kết thúc: TN03 đạt 217,52 đơn/giờ (6 đơn / 99,30 s). Với cùng sáu đơn, nếu tính đến lúc các robot về đủ trạm như TN07, giá trị còn 130,99 đơn/giờ (6 đơn / 164,90 s). Đây chỉ là chỉ số quy đổi cho phiên thử, không phải năng suất ổn định dài hạn của kho.

5.2.3. Đánh giá nhường đường, an toàn và phục hồi

N	Bị AC chặn (robot·s)	Dừng (robot·s)	Không tuyến (robot·s)	Lập đường (thử/đạt)	Vòng tránh (thử/đạt)	Qu t 360°	Bur bị từ chối
1	0	3,40	0	0/0	0/0	0	0
2	8,70	23,70	0	2/1	0/0	0	0
3	25,40	56,80	0	2/1	0/0	0	0
4	2,00	5,60	0	0/0	0/0	0	0
5	0	27,40	24,30	9/1	4/1	1	0
6	74,70	148,00	0	14/12	1/0	0	24
7	35,20	74,60	0	3/2	0/0	0	18

Bảng 5.2. Chỉ số điều khiển bổ sung; thời gian cộng theo robot·giây (không phải thời gian đồng hồ).



Hình 5.2. Số sự kiện bắt đầu bị robot khác chặn ở các kịch bản.

5.2.4. Phân tích các kịch bản đặc trưng

TN04 (đối đầu): hoàn tất 2/2 nhiệm vụ, ghi nhận 3 xung đột và 2,00 robot·giây bị robot khác chặn. Khoảng cách nhỏ nhất tại mẫu là 1,0002 m, lớn hơn ngưỡng 0,98 m.

TN05 (mất tuyến): bộ thử tạo vách chắn rồi tháo ở t = 30 s. Robot có 24,30 robot·giây không tuyến, 9 lần thử lập đường, 4 lần thử vòng tránh và 1 lần quét 360°. Quét không trực tiếp tạo tuyến mới; robot tiếp tục sau khoảng 1,20 s kể từ lúc tháo chắn.

TN06 (hàng đợi): hoàn thành 12/12 đơn, ghi nhận 56 xung đột, 14 lần thử lập lại tuyến (12 thành công) và 24 bước bị kiểm tra an toàn từ chối. TN07 (về sạc): giao đủ 6 đơn ở giây 99,30; đến 164,90 s cả sáu robot đã về đúng ô sạc, trong đó 2 đang CHARGE và 4 đã IDLE.

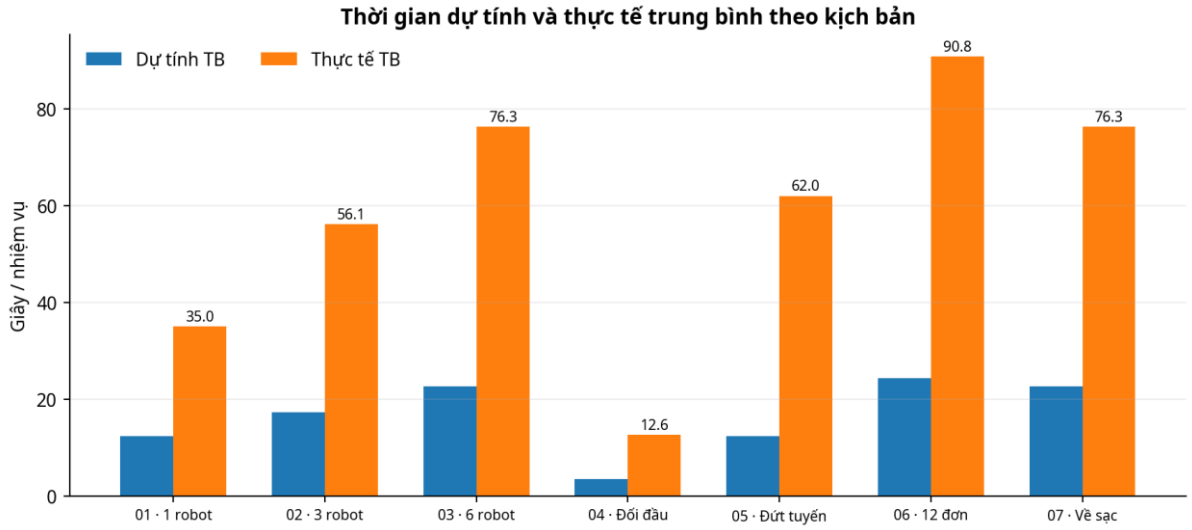
Các kịch bản đa robot có 0 mẫu đo vi phạm 0,98 m, nhưng chỉ số xung đột là lượt bị chặn, KHÔNG phải số va chạm. Mã hiện chưa có bộ đo tiếp xúc hình học độc lập; đo cách nhau 0,1 s chưa bảo đảm không va chạm giữa hai mẫu.

5.2.5. Phân tích khoảng cách an toàn

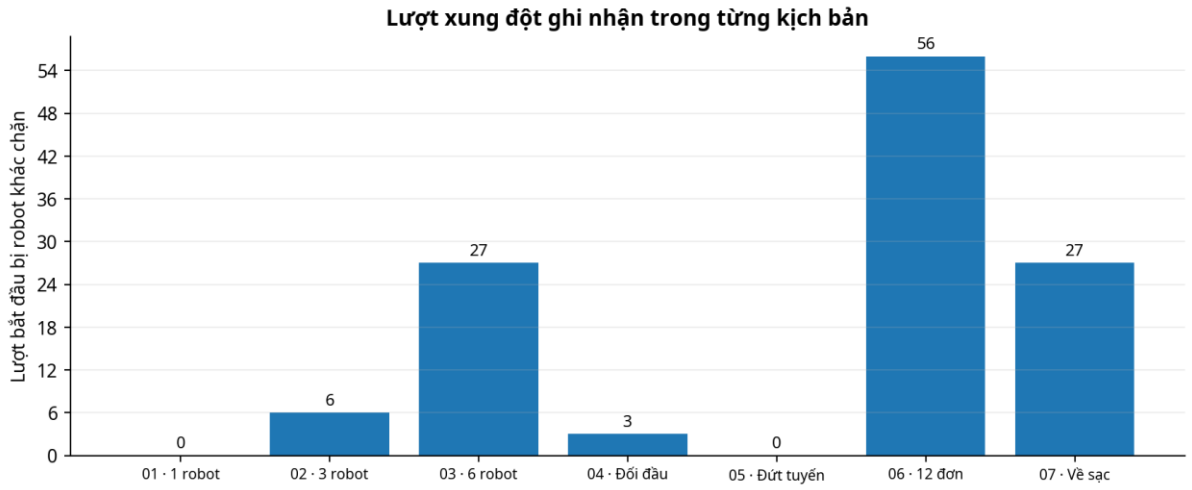
Khoảng cách lấy mẫu nhỏ nhất ở TN02, TN03, TN04, TN06, TN07 lần lượt là 1,0531; 1,0012; 1,0002; 0,9810; 0,9804 m. TN06–TN07 tiến rất sát ngưỡng SAFE_GAP = 0,98 m. Vì vậy, cần bổ sung ghi nhận khoảng cách liên tục và kiểm tra thân robot trước khi kết luận mô hình an toàn khi mật độ tăng.

Mọi kịch bản mới được chạy một lần; chưa có độ lệch chuẩn, khoảng tin cậy hoặc cặp đối chứng chỉ thay đổi một thành phần điều khiển. Những kết quả này có ý nghĩa kiểm chứng chức năng, chưa đủ kết luận "A*/CPA tối ưu hơn bao nhiêu phần trăm".

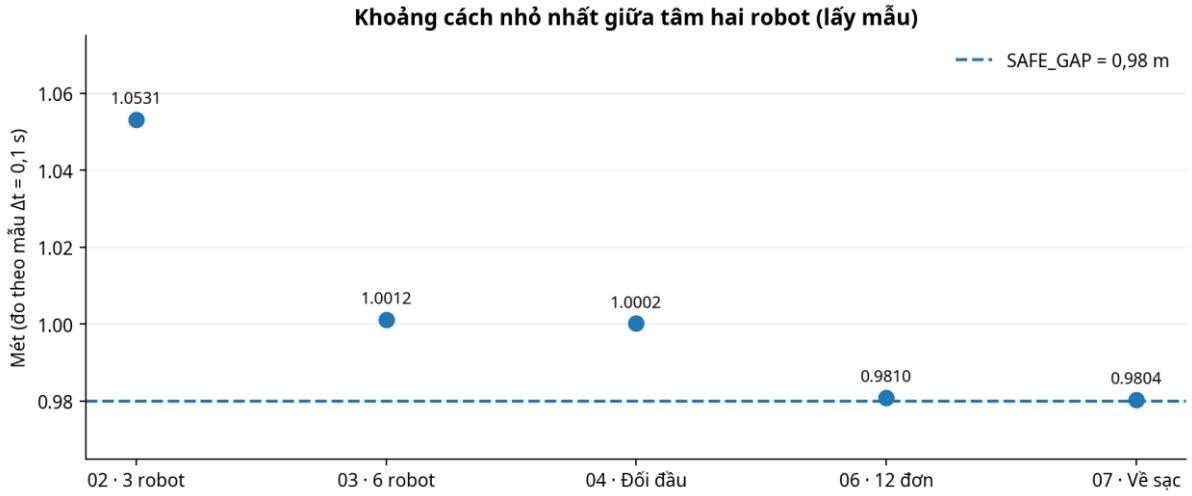
5.2.6. Hình ảnh kết quả thử nghiệm



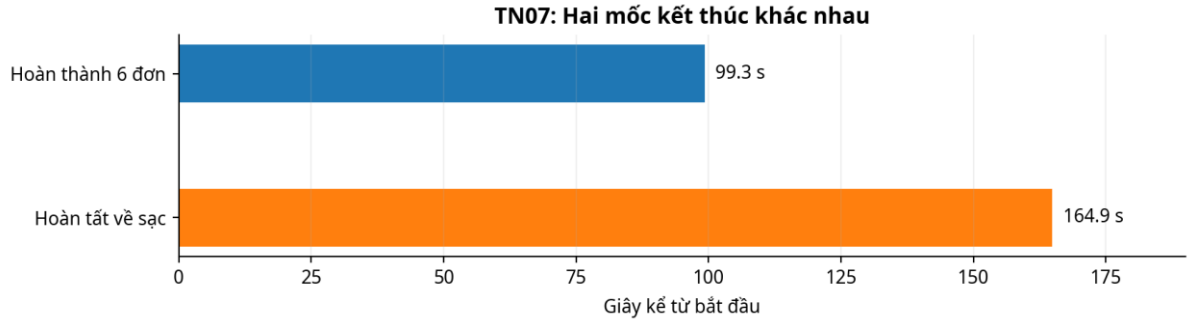
Hình 5.3. Thời gian dự tính và thời gian thực tế trung bình theo kịch bản baseline V21.



Hình 5.4. Số lượt bắt đầu bị robot khác chặn trong từng kịch bản baseline V21.



Hình 5.5. Khoảng cách nhỏ nhất giữa tâm hai robot so với SAFE_GAP = 0,98 m.



Hình 5.6. Hai mốc kết thúc của TN07: hoàn thành đơn và hoàn tất về sạc.

5.3. Đánh giá tổng quan mô hình

5.3.1. Vấn đề giải quyết được

Kết quả thử nghiệm xác nhận trong phạm vi các kịch bản: robot hoàn thành tác vụ cơ sở; nhiều AGV có thể cùng vận hành và nhường đường; bộ điều khiển có cơ chế từ chối bước đi không an toàn; robot có thể tiếp tục sau khi đường hợp lệ trở lại; hàng đợi riêng xử lý được 12 đơn; sáu robot quay về đúng các ô sạc.

5.3.2. Cách giải quyết vấn đề

Vấn đề	Cơ chế đã triển khai trong V21	Bằng chứng từ phép thử
Kệ hàng cản đườ	A* trên lưới, ô không đi qua	Các tuyến nhiệm vụ hoàn thành TN0 TN03
Đường gần ro khác	Chi phí tránh robot, dự đoán vị trí	TN02, TN03 có xung đột và lập lại tuyến
Hai robot tiếp cậ	CPA, cảm biến ảo, giảm tốc theo phanh	TN04 ghi nhận 3 lần bắt đầu bị chặn
Bước di chuy nguy cơ	f5AllowedMove() kiểm tra đoạn tiến	TN06 từ chối 24 bước; TN07 từ chối 18

Vấn đề	Cơ chế đã triển khai trong V21	Bằng chứng từ phép thử
Robot mất đường	Tính lại tuyến, vòng tránh, quét 360°	TN05 phục hồi sau khi vách chắn được th
Đơn hàng liên tiế	Hàng đợi jobQueue riêng theo robot	TN06 hoàn tất 12/12 đơn
Về ô sạc	Đường RETURN, DOCK_TURN, CHAR	TN07 đủ 6/6 robot vào ô sạc

5.3.3. Vấn đề chưa giải quyết được

• Chưa có đặt chỗ ô lưới theo thời gian (MAPF) và chưa bảo đảm thoát mọi loại bế tắc. TN05 chỉ phục hồi khi vách chắn tạm được tháo ra.

• Chưa có bộ đo va chạm độc lập; khoảng cách lấy mẫu chỉ phản ánh các thời điểm đo. Không sử dụng cột collisions = 0 làm bằng chứng "không va chạm".

• Chưa tối ưu tự động phân công robot theo vị trí hàng, pin và tải nhiệm vụ; mô hình pin, nâng hạ và động lực học thực còn đơn giản.

• Chưa chạy xác minh nguyên hệ Colab/Three.js với SKU Kaggle trong bộ thử này; mỗi kịch bản chỉ một lượt, chưa có sai số thống kê và đối chứng bật/tắt từng thuật toán.

Đánh giá mức độ hiệu quả cần tách "đã có cơ chế trong code" khỏi "đã chứng minh cải thiện bằng thực nghiệm". Cả hai không đồng nghĩa.

5.3.4. Phạm vi áp dụng kết luận

TN01–TN07 chủ yếu kiểm tra logic điều khiển robot với bố cục kệ được dựng lại và các tọa độ/SKU thử nghiệm. Sự thành công trong các tình huống này không thay thế kiểm tra hành vi của giao diện FPS, Tablet WMS, tác vụ FEFO trên dữ liệu Kaggle hoặc cảm biến của robot vật lý. Khi mã notebook thay đổi, cần chạy đối chiếu lại bộ thử.

Đặc biệt, TN05 không chứng minh robot tự giải quyết hoàn toàn bế tắc trong một hành lang bị chặn vĩnh viễn: tình huống được giải phóng nhờ thao tác tháo vách chắn ở giây 30. TN07 kiểm chứng quy trình về sạc sau đơn cuối, không phải thử nghiệm một tập đơn độc lập với TN03.

5.4. Định hướng phát triển

5.4.1. Giai đoạn ngắn hạn – hoàn thiện kiểm chứng

Bổ sung bộ phát hiện va chạm hình học độc lập, ghi log từng bước thời gian; xuất thời gian đợi trong hàng, tại kệ, khi vận chuyển và lúc RETURN. Chạy trực tiếp Colab–Three.js với danh sách SKU cố định; thử lặp tối thiểu 10 lần hoặc nhiều bộ tình huống khác nhau, lưu seed và cấu hình.

5.4.2. Giai đoạn trung hạn – tối ưu điều phối

Thiết lập thử nghiệm đối chứng A* có/không có chi phí tránh robot, bật/tắt CPA và cơ chế vòng tránh. Nghiên cứu Cooperative A* hoặc Conflict-Based Search có đặt chỗ theo thời gian, quyền ưu tiên giao lộ và vùng nhường đường. Xây dựng bộ phân công theo khoảng cách, độ ưu tiên FEFO, pin và độ dài hàng đợi.

5.4.3. Giai đoạn dài hạn – tiếp cận thực tế

Nâng cấp mô hình pin và động lực học theo tải trọng, gia tốc; bổ sung mô phỏng cơ cấu lấy hàng, cảm biến LiDAR/camera có nhiễu, độ trễ điều khiển; lưu dữ liệu vào cơ sở dữ liệu, xây dựng dashboard và kiểm thử trong môi trường AGV có giám sát.

5.4.4. Ma trận ưu tiên thực hiện

Ưu tiên	Hạng mục	Tiêu chí nghiệm thu
1	Đo va chạm + log thống nhất	Có vị trí, thời điểm, mức độ và file dữ liệu thô
2	Thử đối chứng A*, CPA, detour	Cùng dữ liệu đầu vào; thống kê chênh lệch và biến thiên
3	Điều phối đa robot có thời gian	Giảm xung đột ở giao lộ/hành lang trên bộ ca thử
4	Phân công nhiệm vụ tự động	Giảm thời gian hoàn thành với cùng danh sách đơn
5	Mô hình năng lượng + cảm biến thực	Có sai số, độ trễ và yêu cầu an toàn được kiểm tra

5.4.5. Đề xuất thí nghiệm tiếp theo

Tiến hành hai bộ đối chứng trên cùng tập vị trí và thứ tự phát lệnh: (i) có/không có chi phí tránh robot trong A*; (ii) có/không có CPA nhưng vẫn giữ lớp f5AllowedMove(). Với mỗi bộ, lưu quãng đường, thời gian, số lượt chặn, dmin và số nhiệm vụ hoàn thành. Thêm kịch bản khóa kín hành lang để đánh giá giới hạn bế tắc một cách trung thực; không tự động giải phóng vật cản nếu mục tiêu là đánh giá tự phục hồi.

5.5. Hồ sơ kiểm chứng

5.5.1. /exam và quy trình kiểm thử

Mỗi lần chạy phải ghi phiên bản notebook, cấu hình đội robot, seed hoặc thứ tự phát lệnh, bộ kịch bản TN01–TN07 và số lần lặp. Một kịch bản đạt khi nhiệm vụ hoàn thành, tuyến không đi qua ô cấm, bước bị từ chối không làm mất trạng thái và robot kết thúc đúng trạng thái dự kiến.

5.5.2. /logs và schema sự kiện

Log tối thiểu gồm timestamp, robot_id, phase, position, speed, event, conflict_count, collision_flag và distance_to_nearest. Lịch sử nhiệm vụ và lịch sử vận tốc được xuất thành CSV; log frame-level là phần cần bổ sung để kiểm tra liên tục khoảng cách và nguyên nhân bị chặn.

5.5.3. /evals và tiêu chí đánh giá

Bộ đánh giá theo dõi completion rate, makespan, ETA error, distance, blocked robot·s, replans, rejected moves, dmin và collisions. Cần tách rõ “có cơ chế trong code” khỏi “đã

chứng minh bằng thử nghiệm”; các kết quả hiện tại chủ yếu là kiểm chứng chức năng một lượt cho mỗi kịch bản.

CHƯƠNG 6: KẾT LUẬN VÀ KIẾN NGHỊ

6.1. Kết luận

Trong bối cảnh các doanh nghiệp ngày càng đẩy mạnh ứng dụng công nghệ vào hoạt động logistics, việc xây dựng hệ thống kho thông minh sử dụng robot AGV đang trở thành xu hướng tất yếu nhằm nâng cao hiệu quả vận hành và giảm chi phí lao động. Đề tài đã tập trung nghiên cứu bài toán tối ưu hóa tuyến đường và giải quyết xung đột trong vận hành AGV trong môi trường kho thông minh.

Mô hình được xây dựng dựa trên nguyên tắc quản lý tồn kho FEFO kết hợp với hệ thống robot AGV hoạt động trong không gian kho 3D. Thuật toán A* được sử dụng để tính toán tuyến đường tối ưu cho robot, trong khi cơ chế CPA và các kỹ thuật điều tốc được áp dụng nhằm giảm thiểu nguy cơ va chạm khi nhiều robot cùng hoạt động.

Kết quả nghiên cứu cho thấy mô hình có khả năng:

Tối ưu hóa tuyến đường di chuyển của robot.

Hỗ trợ quản lý hàng hóa theo nguyên tắc FEFO.

Giảm nguy cơ xung đột trong quá trình vận hành.

Tăng tính tự động hóa cho nhà kho.

Nâng cao khả năng giám sát và quản lý hoạt động kho hàng.

Mặc dù mới dừng ở mức mô phỏng, mô hình đã thể hiện tiềm năng ứng dụng trong việc xây dựng các hệ thống kho tự động hiện đại phục vụ cho doanh nghiệp logistics và thương mại điện tử.

6.2. Định hướng phát triển

Các ưu tiên phát triển được xếp theo ba bước. Trước hết cần bổ sung bộ đo va chạm độc lập và lặp lại các kịch bản với seed cố định. Tiếp theo có thể thử MAPF có đặt chỗ theo thời gian và tối ưu phân công theo khoảng cách, FEFO, pin và hàng đợi. Sau cùng, mô hình có thể mở rộng với động lực học, cảm biến và tích hợp WMS/ERP.

TÀI LIỆU THAM KHẢO